



# 卷积神经网络的应用与实践

## CNN Application & Practice



- 01 卷积网络应用概述 (CNN Applications)**
- 02 图像分类与检测(Classification & Detection)**
- 03 实践(Practice)**





PART ONE

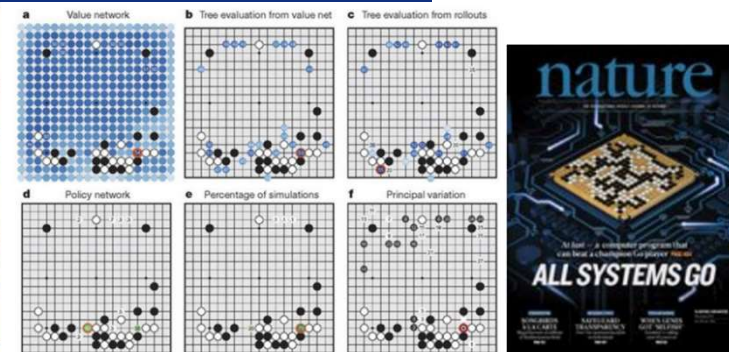
## 卷积网络应用概述

---

## CNN Applications



## ■ AlphaGO



The input to the policy network is a  $19 \times 19 \times 48$  image stack consisting of 48 feature planes. The first hidden layer zero pads the input into a  $23 \times 23$  image, then convolves  $k$  filters of kernel size  $5 \times 5$  with stride 1 with the input image and applies a rectifier nonlinearity. Each of the subsequent hidden layers 2 to 12 zero pads the respective previous hidden layer into a  $21 \times 21$  image, then convolves  $k$  filters of kernel size  $3 \times 3$  with stride 1, again followed by a rectifier nonlinearity. The final layer convolves 1 filter of kernel size  $1 \times 1$  with stride 1, with a different bias for each position, and applies a softmax function. The match version of AlphaGo used  $k = 192$  filters; Fig. 2b and Extended Data Table 3 additionally show the results of training with  $k = 128, 256$  and  $384$  filters.

### policy network:

[19x19x48] Input

CONV1: 192 5x5 filters, stride 1, pad 2 => [19x19x192]

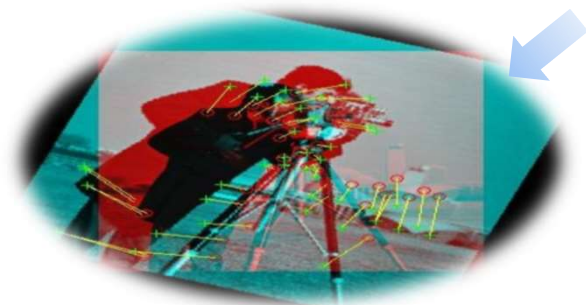
CONV2..12: 192 3x3 filters, stride 1, pad 1 => [19x19x192]

CONV: 1 1x1 filter, stride 1, pad 0 => [19x19] (*probability map of promising moves*)

- 分布式系统: 1202 个CPU 和176 块GPU
- 单机版: 48 个CPU 和8 块GPU
- 走子速度: 3 毫秒-2 微秒



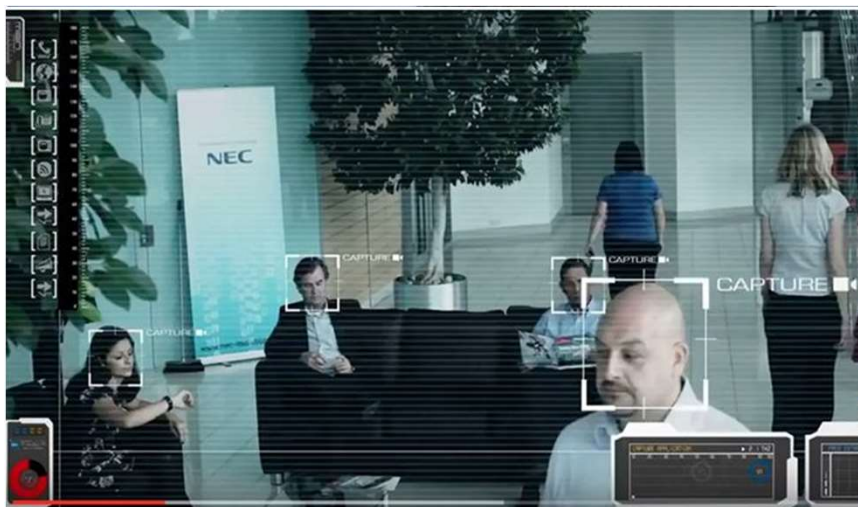
# 自然语言处理



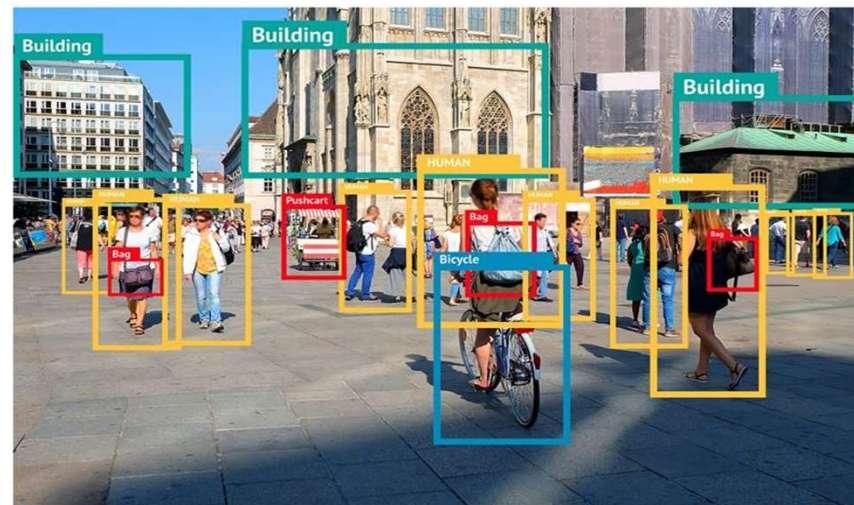
# 医学影像分析



## ■ 计算机视觉领域

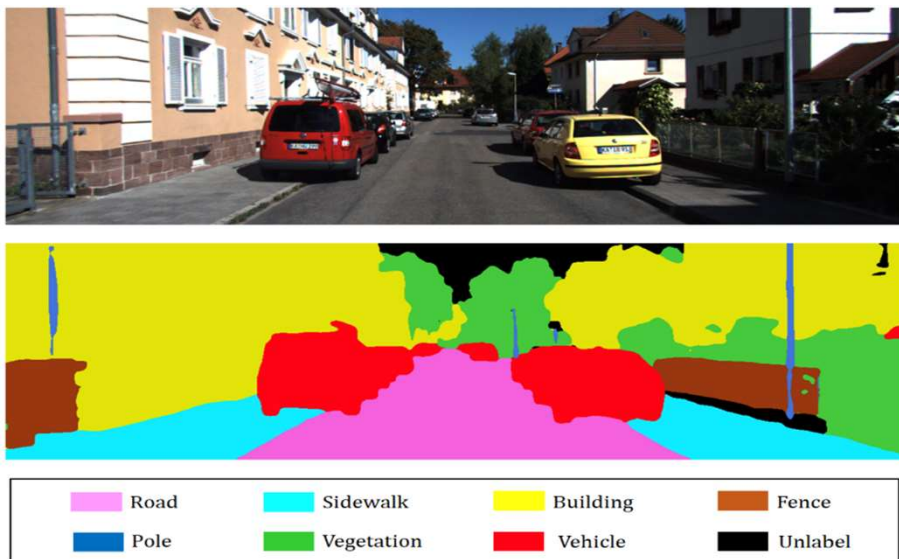


**人脸识别：**定位人脸位置，识别身份，常用于机场车站的安检，刑事侦查等领域。



**目标检测：**检测场景中所有目标，确定其位置、大小与类别，可为后续行为分析、场景理解等服务

## ■ 计算机视觉领域



**语义分割**：对图像或视频中各像素点的语义类别标签（如天空、楼房、草地）进行预测，该技术可服务于计算机的场景理解、无人车驾驶等。



**人体姿态估计**：定位人在图像中的位置，检测人体各个关键点的位置，可服务于人机交互



## ➤ 图像处理领域



**图像去雾：**对有雾图像进行处理，以恢复出原始场景的颜色和能见度



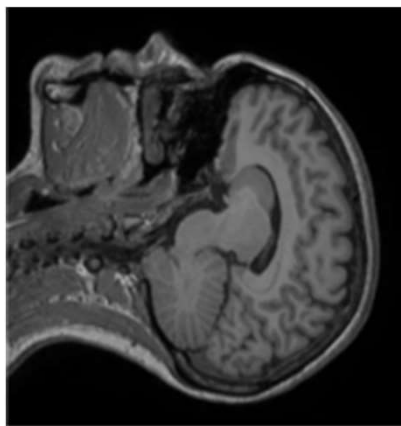
**图像去雨：**去除雨中拍摄图像的雨滴，并保持图像原有的纹理细节

## ➤ 图像处理领域

原始有噪图像



去噪结果



原始图像



预测蒙版



背景替换

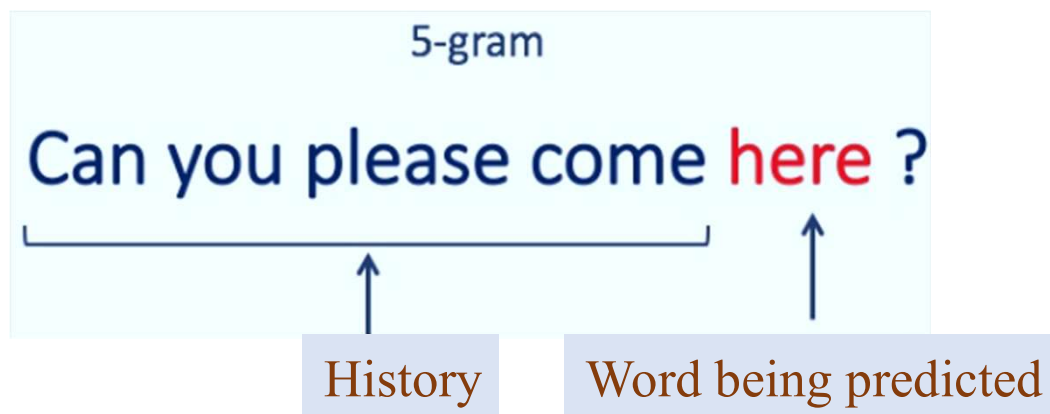


**医学图像去噪**：去除图像噪声，尽可能恢复图像的纹理细节，可以帮助医学工作者，做出更有效的诊断

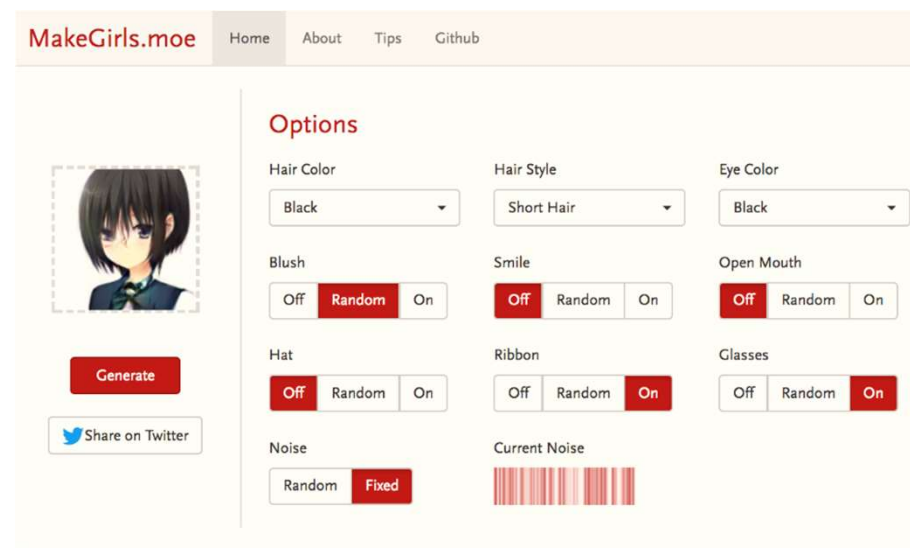
**抠图**：精确地预测出图像中前景的蒙版（透明度）。可应用于图像前景与背景的编辑、影像后期处理等



## ■ 自然语言处理领域



语言模型根据前几个单词输入进行分析，并预测下一个词



文本图像合成：根据文本生成理想的图片, 人人都可进行艺术创作

## ■ 自然语言处理领域



Speech

Text

**语音识别**：无论是手机打字时的语音输入，还是iPhone的智能助力SIRI，当我们使用语音和机器进行交互时，背后就有语音识别算法的支持

### Article

Computational complexity theory is a branch of the theory of computation in theoretical computer science that focuses on classifying computational problems according to their inherent difficulty, and relating those classes to each other. A computational problem is understood to be a task that is in principle amenable to being solved by a computer, which is equivalent to stating that the problem may be solved by mechanical application of mathematical steps, such as an algorithm.

### Question

By what main attribute are computational problems classified utilizing computational complexity theory?

Ground Truth Answers: inherent difficulty their inherent difficulty inherent difficulty

Prediction: inherent difficulty

**自动问答**：计算机在阅读一段文字之后，回答一些与文字相关的问题



## PART TWO

## 分类与检测 Classification & Detection



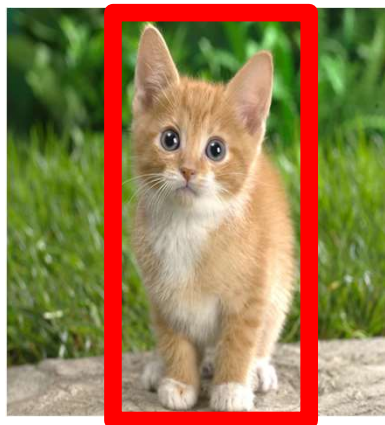
## ■ 计算机视觉中的几个重要而基础任务

**Classification**



CAT

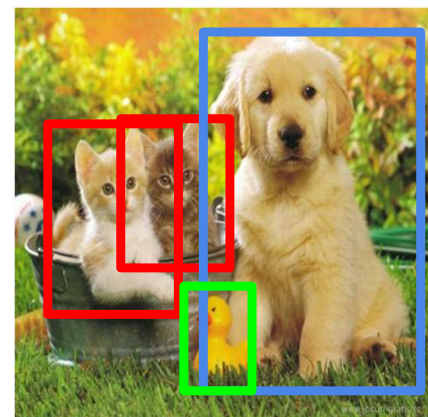
**Classification  
+ Localization**



CAT

Single object

**Object  
Detection**



CAT, DOG, DUCK

Multiple objects



**Classification**



CAT

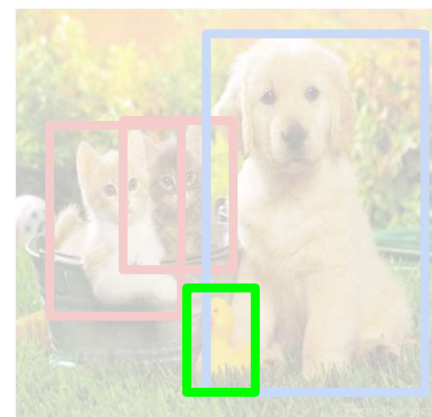
**Classification  
+ Localization**



CAT

Single object

**Object  
Detection**



CAT, DOG, DUCK

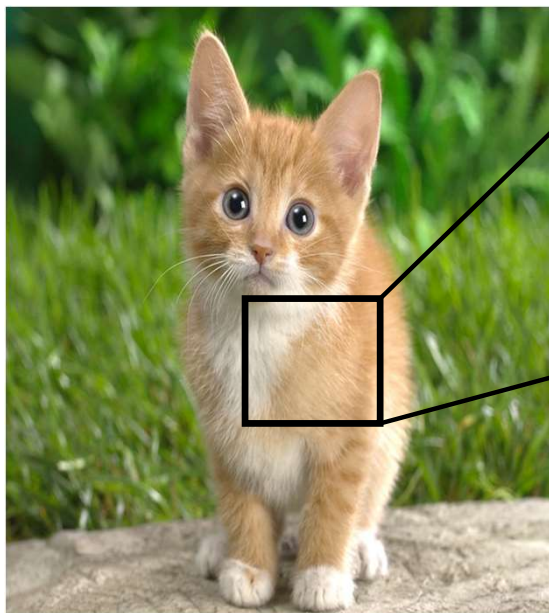
Multiple objects

- 假设给定一系列类别标签集合: {dog, cat, truck, plane, ...}



→ cat

## ➤ 人与计算机之间存在语义鸿沟



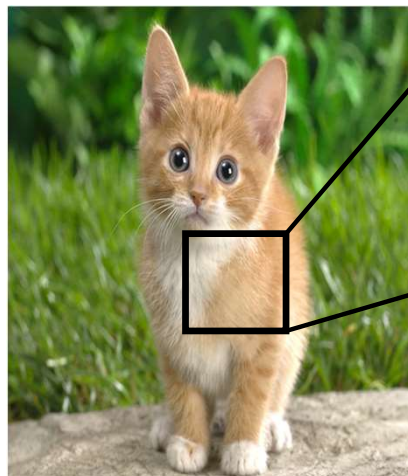
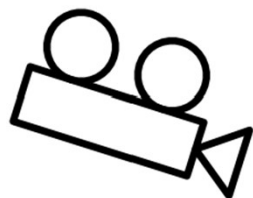
```
[[105 112 108 111 104 99 106 99 96 103 112 119 104 97 93 87]  
[ 91 98 102 106 104 79 98 103 99 105 123 136 110 105 94 85]  
[ 76 85 90 105 128 105 87 96 95 99 115 112 106 103 99 85]  
[ 99 81 81 93 120 131 127 100 95 98 102 99 96 93 101 94]  
[106 91 61 64 69 91 88 85 101 107 109 98 75 84 96 95]  
[114 108 85 55 55 69 64 54 64 87 112 129 98 74 84 91]  
[133 137 147 103 65 81 80 65 52 54 74 84 102 93 85 82]  
[128 137 144 140 109 95 86 70 62 65 63 63 60 73 86 101]  
[125 133 148 137 119 121 117 94 65 79 80 65 54 64 72 98]  
[127 125 131 147 133 127 126 131 111 96 89 75 61 64 72 84]  
[115 114 109 123 150 148 131 118 113 109 100 92 74 65 72 78]  
[ 89 93 90 97 108 147 131 118 113 114 113 109 106 95 77 80]  
[ 63 77 86 81 77 79 102 123 117 115 117 125 125 130 115 87]  
[ 62 65 82 89 78 71 80 101 124 126 119 101 107 114 131 119]  
[ 63 65 75 88 89 71 62 81 120 138 135 105 81 98 110 118]  
[ 87 65 71 87 106 95 69 45 76 130 126 107 92 94 105 112]  
[118 97 82 86 117 123 116 66 41 51 95 93 89 95 102 107]  
[164 146 112 80 82 120 124 104 76 48 45 66 88 101 102 109]  
[157 170 157 120 93 86 114 132 112 97 69 55 70 82 99 94]  
[130 128 134 161 139 100 109 118 121 134 114 87 65 53 69 86]  
[128 112 96 117 150 144 120 115 104 107 102 93 87 81 72 79]  
[123 107 96 86 83 112 153 149 122 109 104 75 80 107 112 99]  
[122 121 102 80 82 86 94 117 145 148 153 102 58 78 92 107]  
[122 164 148 103 71 56 78 83 93 103 119 139 102 61 69 84]]
```

计算机所看到的信息形式

图像仅为一个 $[0, 255]$ 的整型张量

e.g.  $800 \times 600 \times 3$  (RGB 3通道)

## ➤ 挑战因素：视角变化



```
[[105 112 108 111 104 99 106 99 96 103 112 119 104 97 93 87]  
[ 91 98 102 106 104 79 98 103 99 105 123 136 110 105 94 85]  
[ 76 85 90 105 128 105 87 96 95 99 115 112 106 103 99 85]  
[ 99 81 81 93 120 131 127 100 95 98 102 99 96 93 101 94]  
[106 91 61 64 69 91 88 85 101 107 109 98 75 84 96 95]  
[114 108 85 55 55 69 64 54 64 87 112 129 98 74 84 91]  
[133 137 147 103 65 81 80 65 52 54 74 84 102 93 85 82]  
[128 137 144 140 109 95 86 70 62 65 63 63 60 73 86 101]  
[125 133 148 137 119 121 117 94 65 79 80 65 54 64 72 98]  
[127 125 131 147 133 127 126 131 111 96 89 75 61 64 72 84]  
[115 114 109 123 150 148 131 118 113 109 100 92 74 65 72 78]  
[ 89 93 90 97 108 147 131 118 113 114 113 109 106 95 77 80]  
[ 63 77 86 81 77 79 102 123 117 115 117 125 125 130 115 87]  
[ 62 65 82 89 78 71 80 101 124 126 119 101 107 114 131 119]  
[ 63 65 75 88 89 71 62 81 120 138 135 105 81 98 110 118]  
[ 87 65 71 87 106 95 69 45 76 130 126 107 92 94 105 112]  
[118 97 82 86 117 123 116 66 41 51 95 93 89 95 102 107]  
[164 146 112 80 82 120 124 104 76 48 45 66 88 101 102 109]  
[157 170 157 120 93 86 114 132 112 97 69 55 70 82 99 94]  
[130 128 134 161 139 100 109 118 121 134 114 87 65 53 69 86]  
[128 112 96 117 150 144 120 115 104 107 102 93 87 81 72 79]  
[123 107 96 86 83 112 153 149 122 109 104 75 80 107 112 99]  
[122 121 102 80 82 86 94 117 145 148 153 102 58 78 92 107]  
[122 164 148 103 71 56 78 83 93 103 119 139 102 61 69 84]]
```

当图像采集设备移动时，  
所有像素值将随之变化



## ➤ 挑战因素：背景杂乱



## ➤ 挑战因素：光照变化





## ➤ 挑战因素：遮挡



➤ 挑战因素：类内差异





## ➤ 图像分类流程



特征提取

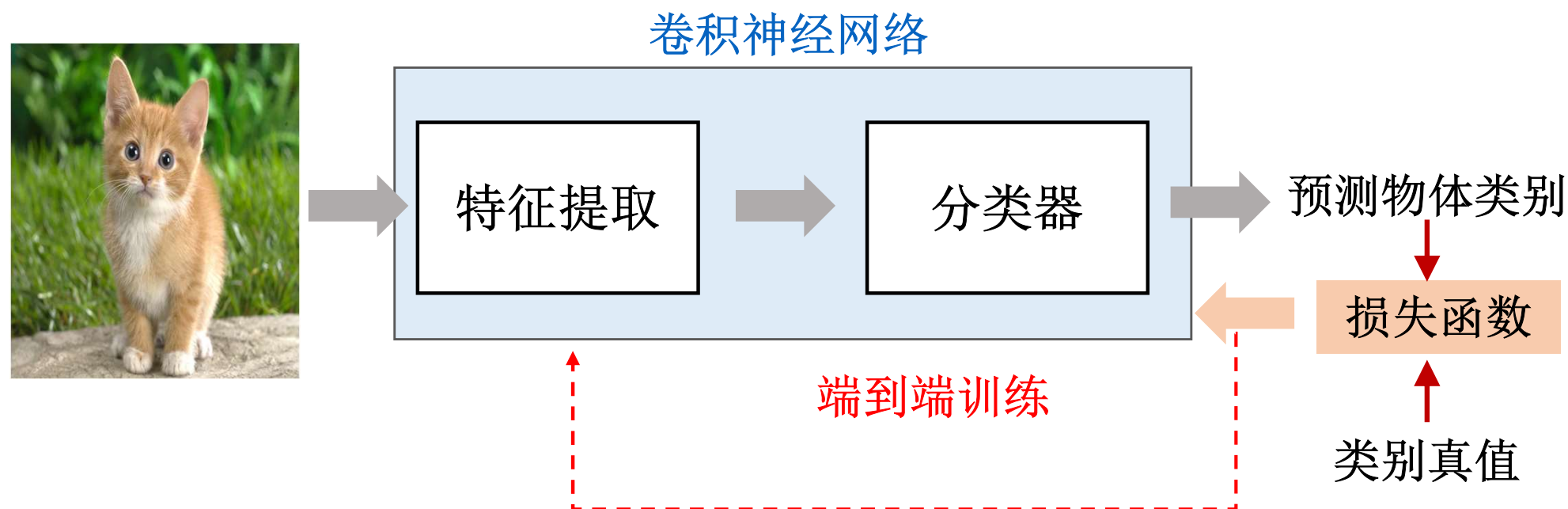


分类器



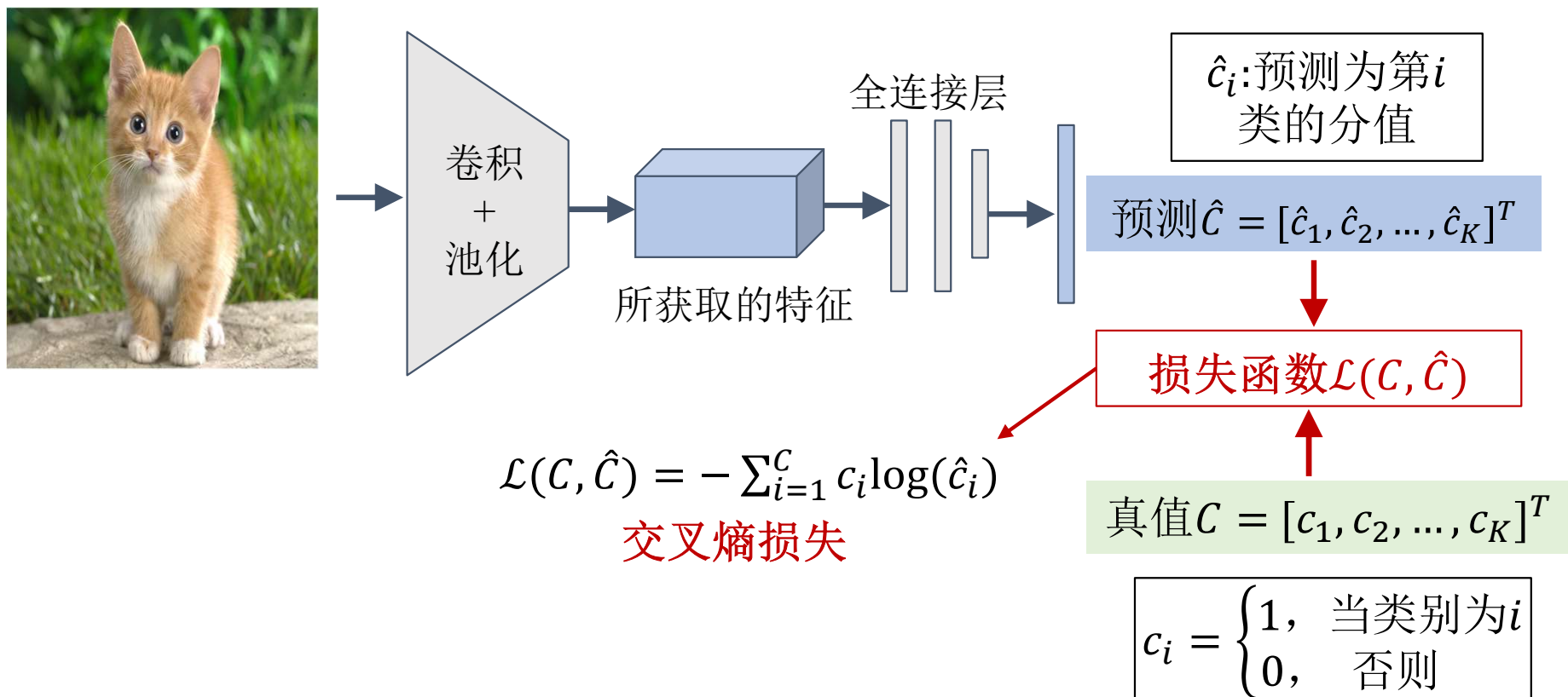
预测物体类别

## ➤ 图像分类流程



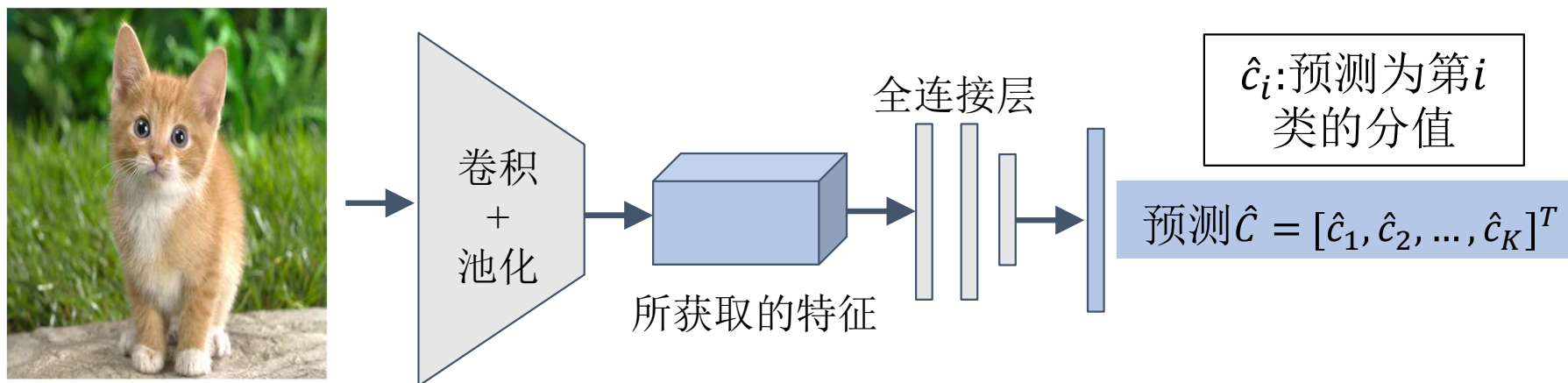
## ➤ 训练阶段

- 假设给定K类



## ➤ 训练阶段

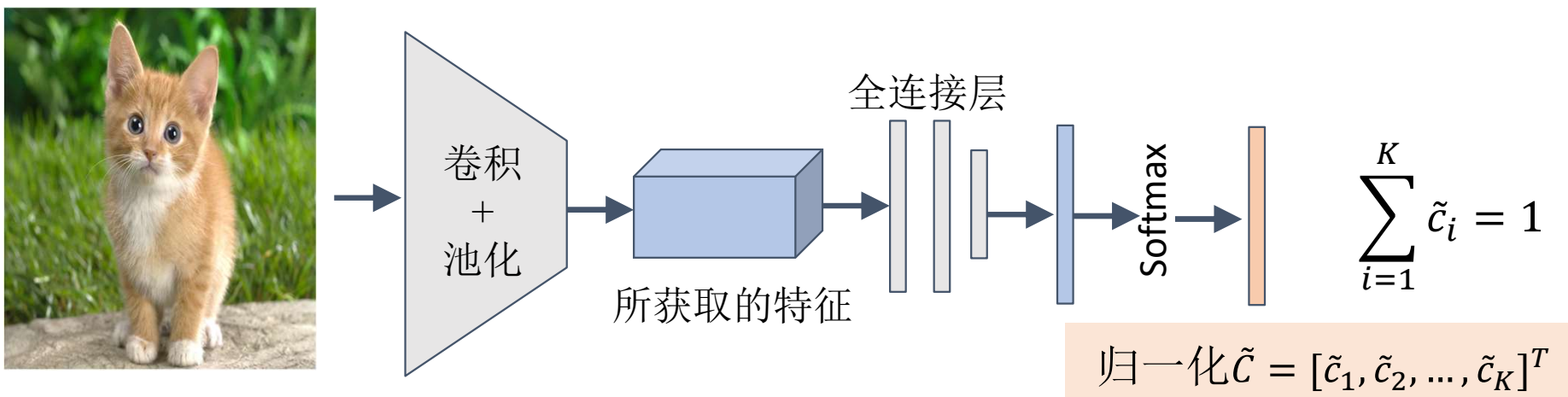
- 假设给定K类





## ➤ 测试阶段

- 假设给定K类



最终预测类别：最大值 $\tilde{c}_i$ 所对应的类别

## ➤ 数据集- MNIST

- 10分类: 0~9
- 训练集: 60000个样本
- 测试集: 10000个样本

### THE MNIST DATABASE

of handwritten digits

Yann LeCun, Courant Institute, NYU

Corinna Cortes, Google Labs, New York

Christopher J.C. Burges, Microsoft Research, Redmond



## ➤ 数据集-PASCAL VOC

20 classes



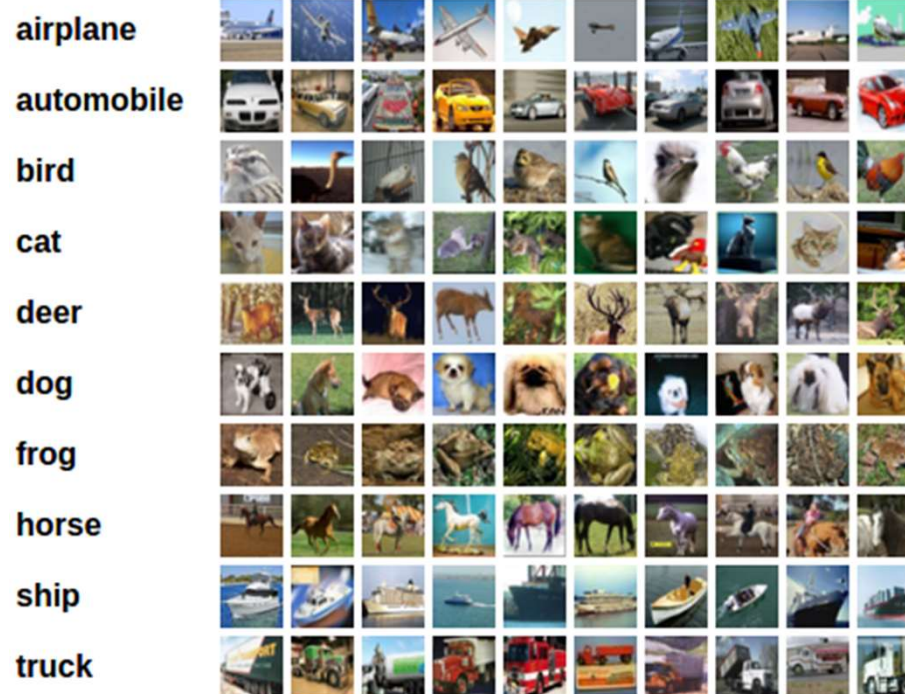
- 包含4大类，20小类

- *Person*: person
- *Animal*: bird, cat, cow, dog, horse, sheep
- *Vehicle*: aeroplane, bicycle, boat, bus, car, motorbike, train
- *Indoor*: bottle, chair, dining table, potted plant, sofa, tv/monitor

## ➤ 数据集- CIFAR-10 & CIFAR-100



- 共**60000**张图像，每个类别6000张，均为大小**32×32**的**RGB**图像
- 训练集：**50000**张图像；测试集：**10000**张图像（每个类别1000张）





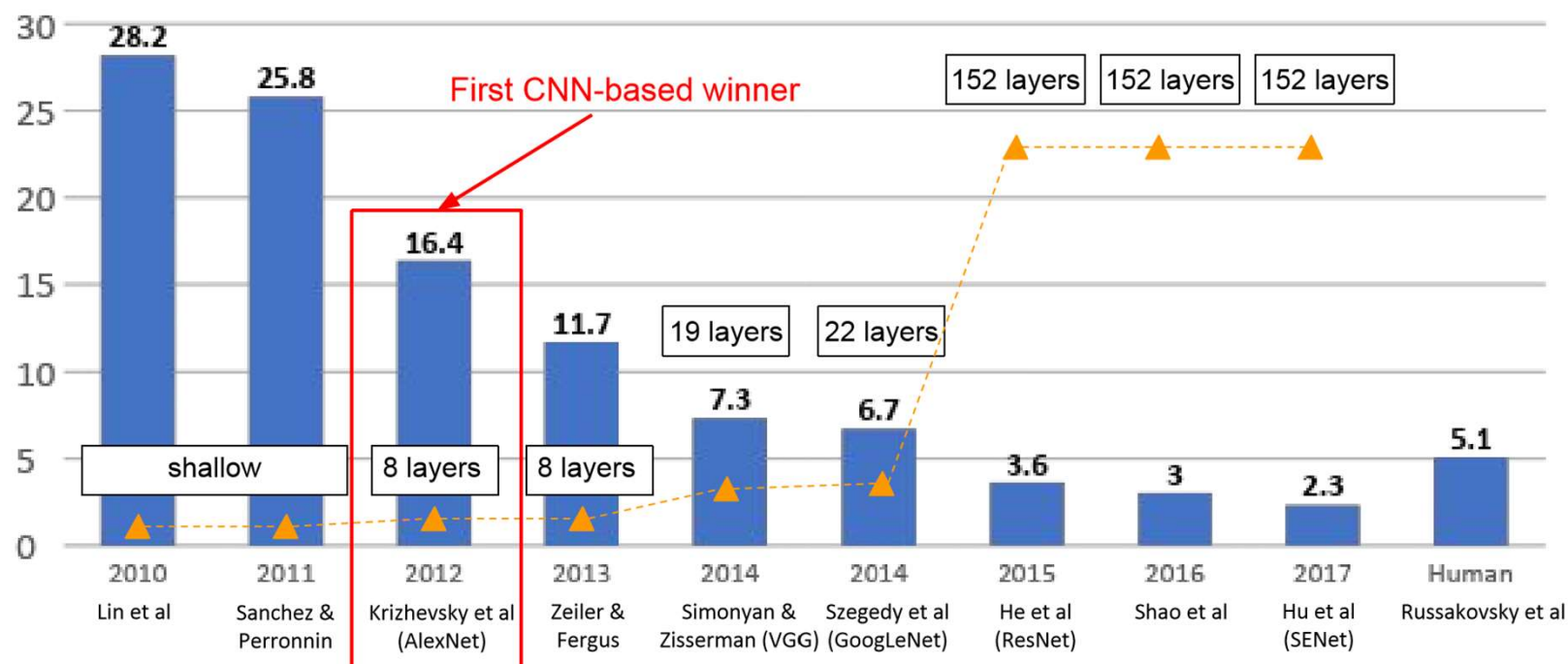
## ➤ 数据集-ImageNet

- 目前世界上图像识别最大的数据库，约**1500**万张图像，**2.2**万类别
- **ISLVR**C为其中的分类任务数据集，约**1.2M**张图像，**1000**个类别



## ➤ 数据集-ImageNet

- 极大地推动了图像分类领域的快速发展

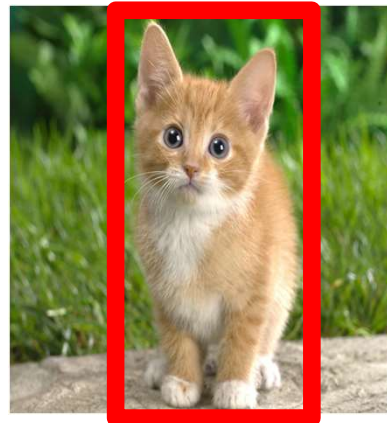


Classification



CAT

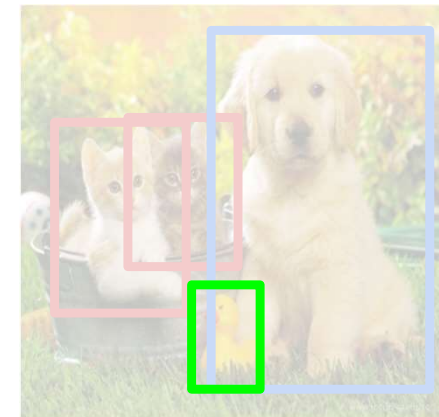
Classification  
+ Localization



CAT

Single object

Object  
Detection



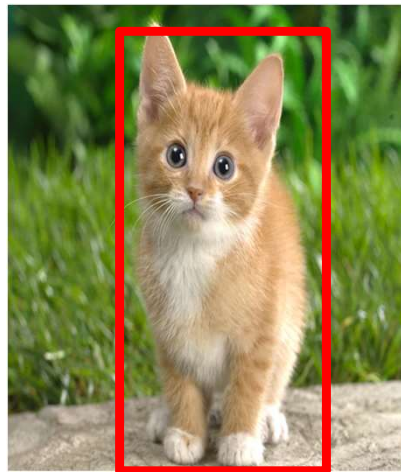
CAT, DOG, DUCK

Multiple objects



## ➤ 目标定位

- 预测图中物体的位置与大小

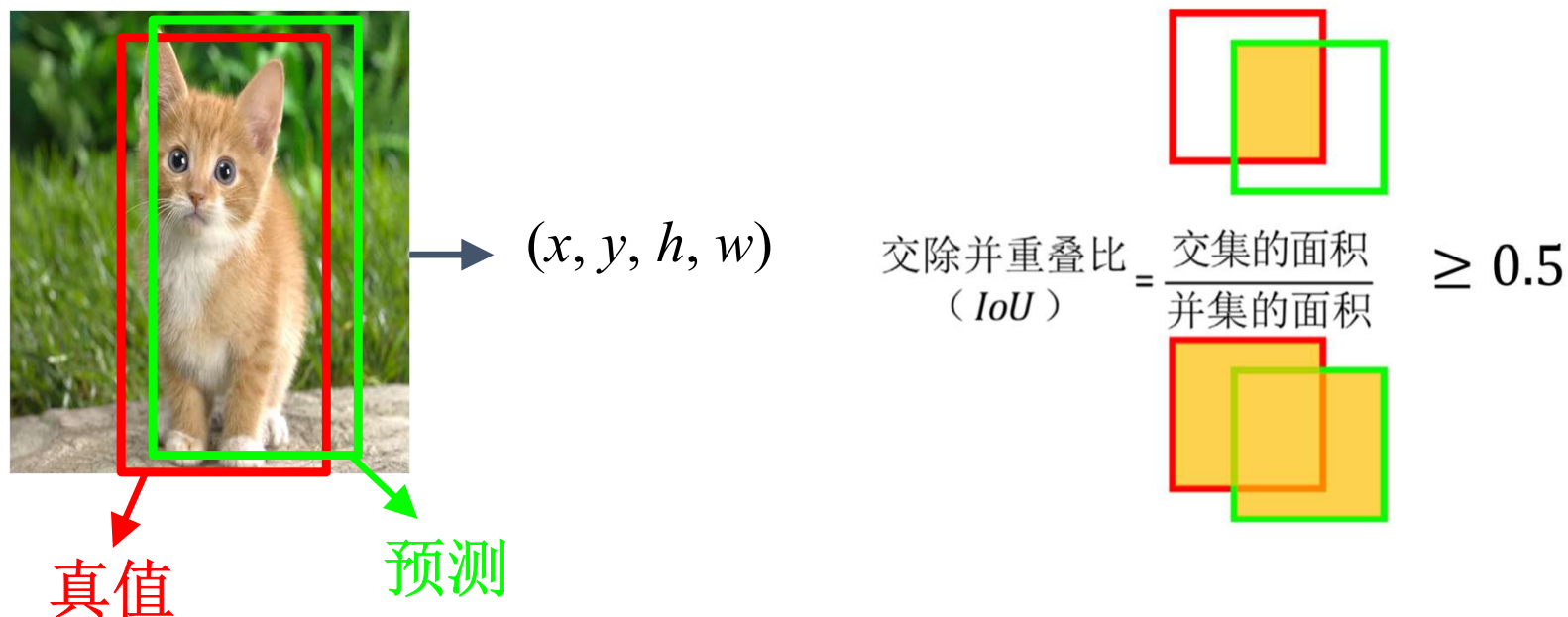


→  $(x, y, h, w)$



## ➤ 目标定位

- 评估指标: 并交比 (**Intersection over Union**)



## ➤ 将定位视为目标框的回归问题



输入: 图像

CNN

输出: 目标框  
 $(\hat{x}, \hat{y}, \hat{h}, \hat{w})$

损失函数

$$\mathcal{L}_{box} = ||x - \hat{x}||_2 + ||y - \hat{y}||_2 + ||h - \hat{h}||_2 + ||w - \hat{w}||_2$$

真值: 目标框  
 $(x, y, h, w)$

## ➤ 分类+定位

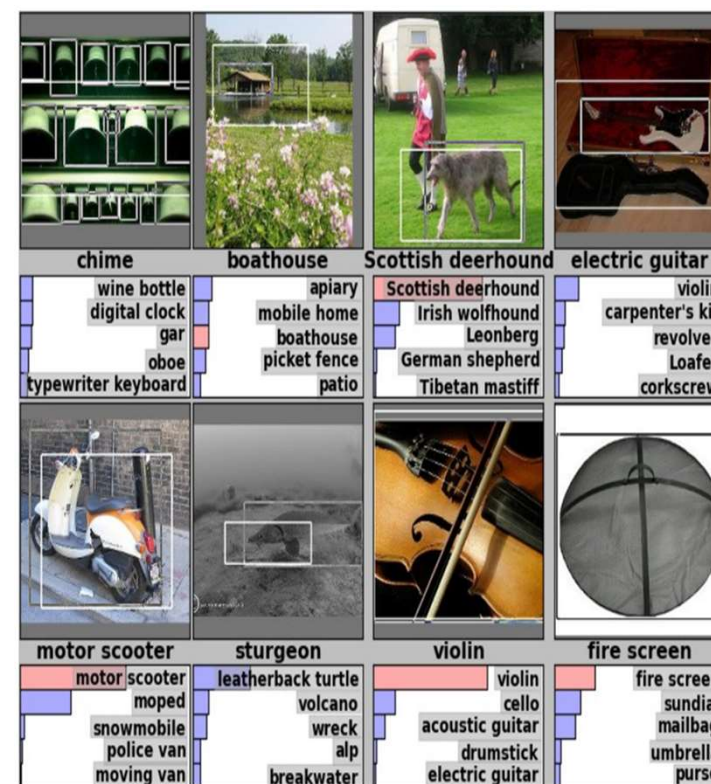
- 预测图像中物体的类别、位置与大小



类别:  $c$   
目标框:  $(x, y, h, w)$

## ➤ 设置描述

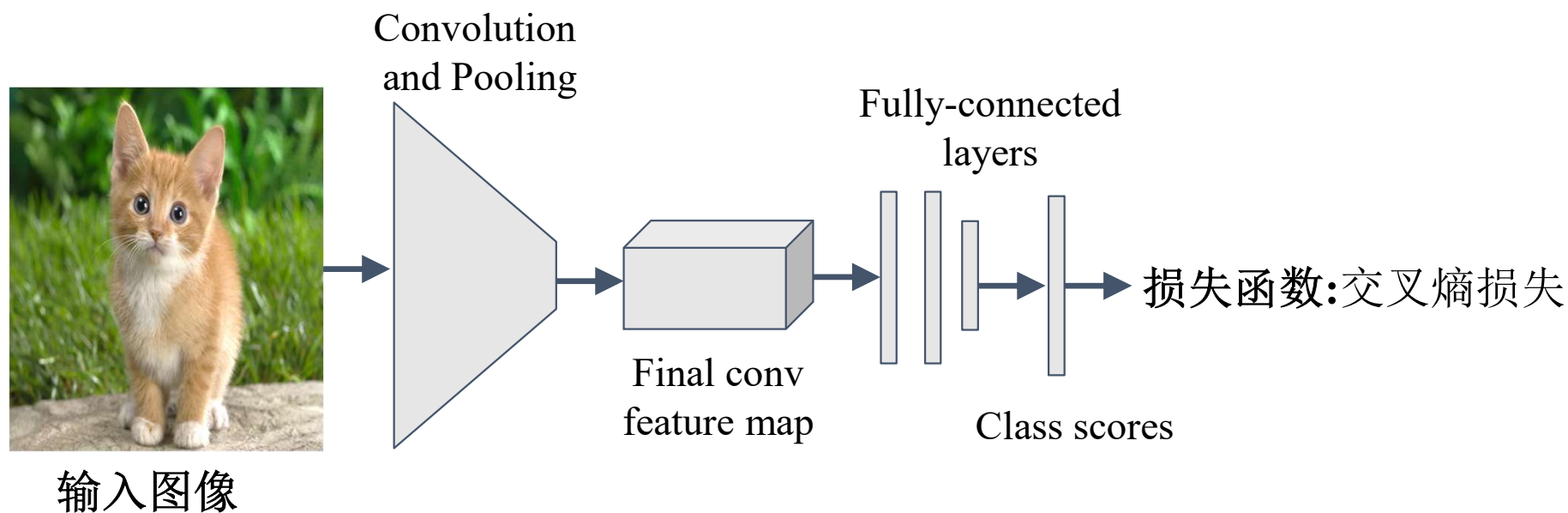
- 图像的真值包括 **类别+目标框** ( $x, y, h, w$ )
- 给定一张图像，网络预测5个值
  - 1: 类别, 4: 目标框
- 预测正确判断:
  - 类别预测正确
  - 目标框的预测与真值的交叉比大于0.5





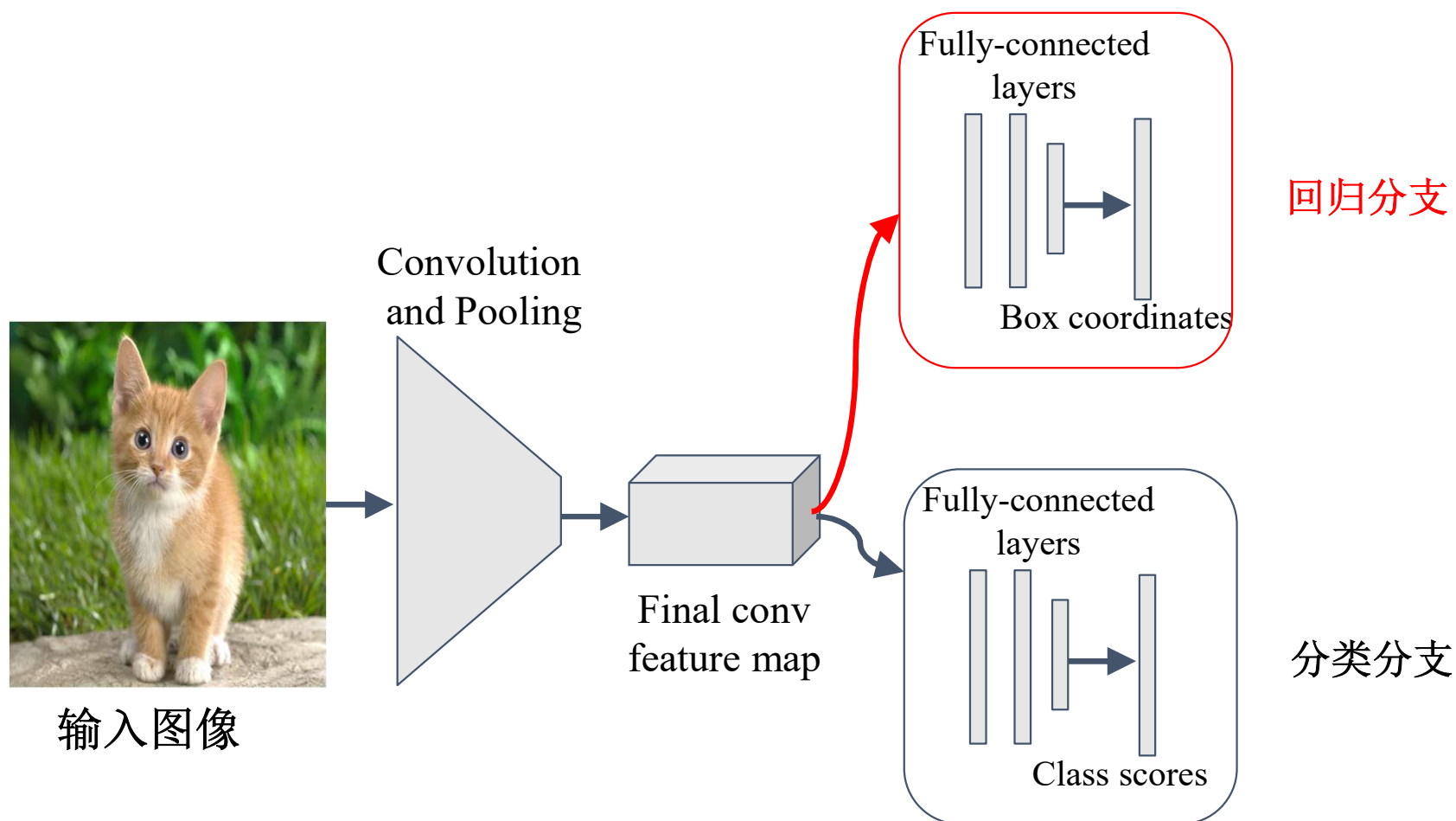
## ➤ 实施步骤

- **Step1:** 训练（或下载）一个分类网络（VGG, GoogLeNet, ResNet）



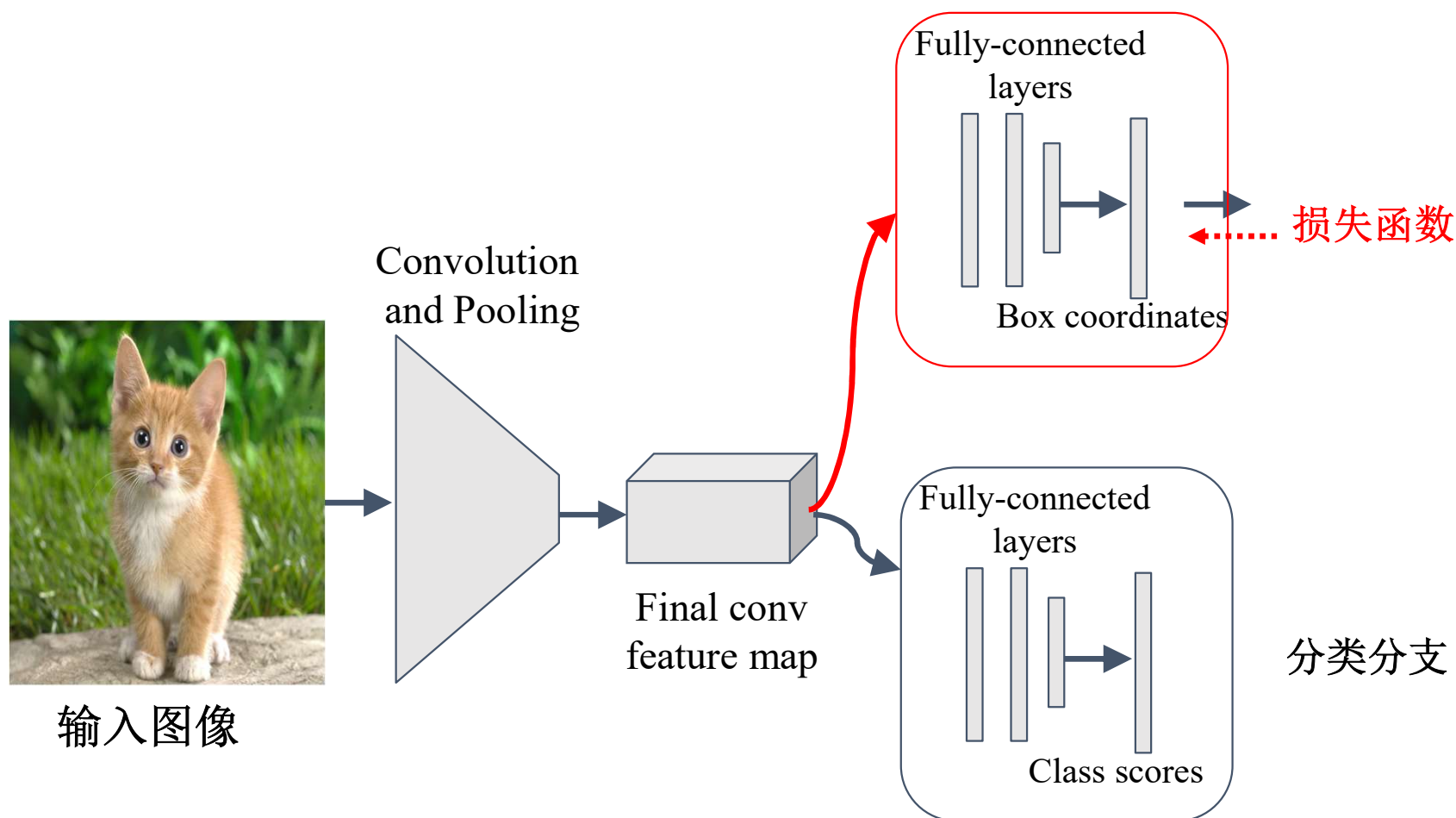
## ➤ 实施步骤

- **Step2:** 在网络末端增加一个新的全连接“回归分支”



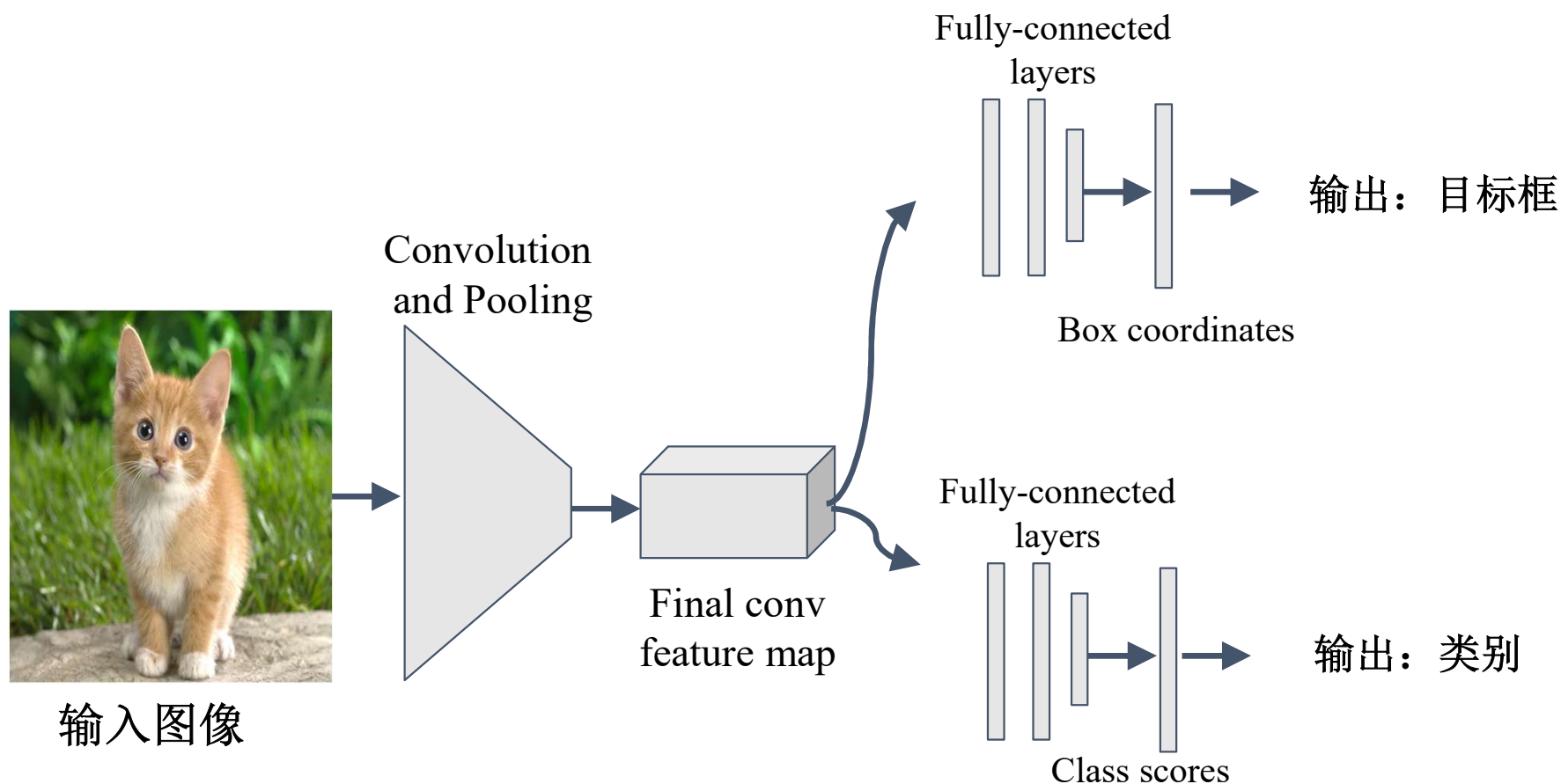
## ➤ 实施步骤

- **Step3:** 训练定位的“回归分支”（或两个分支联合训练）



## ➤ 实施步骤

- **Step4:** 测试时，两个分支均进行预测





Classification



CAT

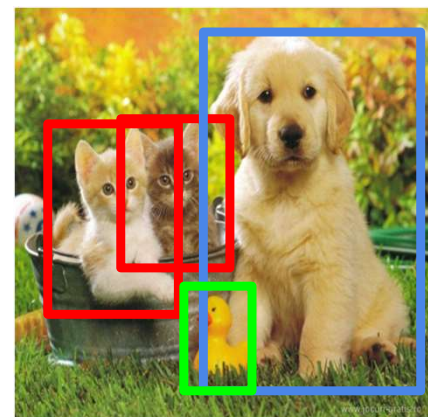
Classification  
+ Localization



CAT

Single object

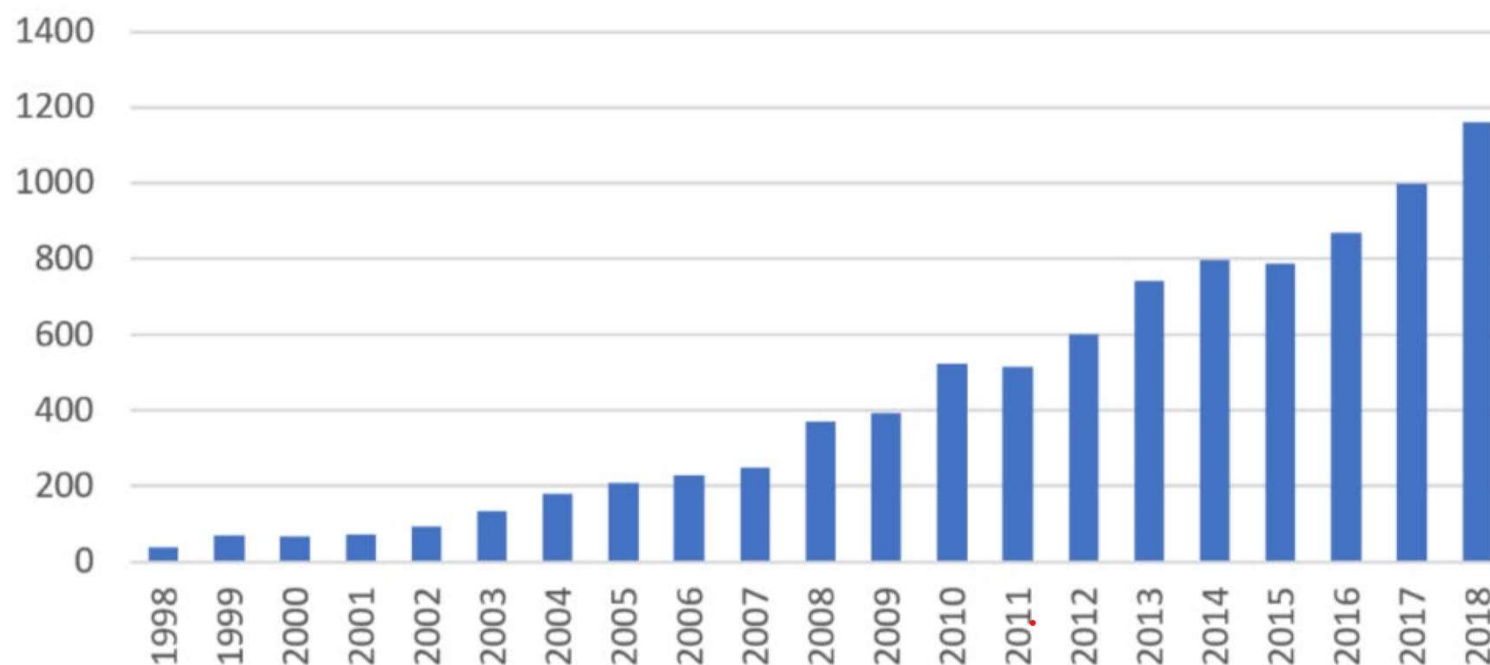
Object  
Detection



CAT, DOG, DUCK

Multiple objects

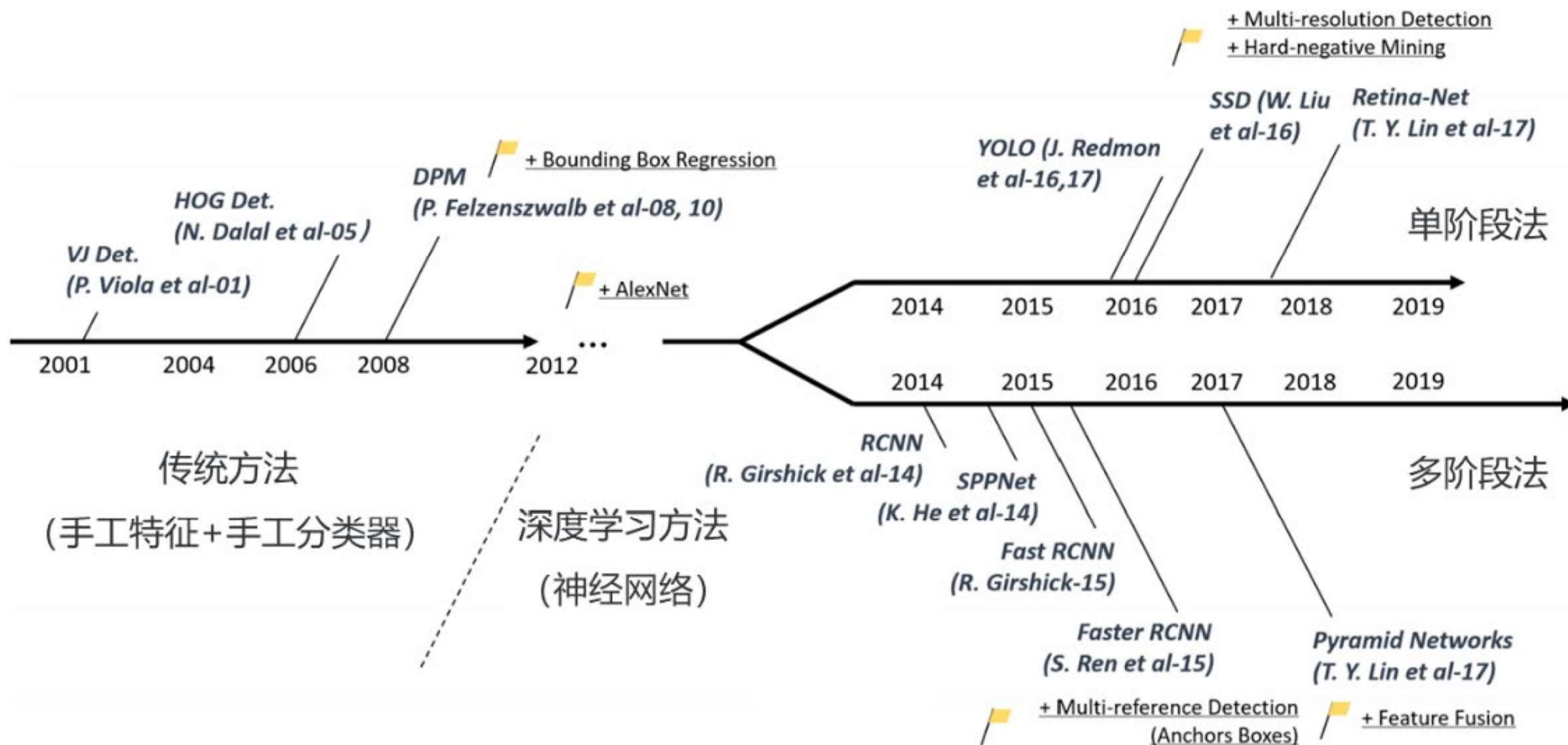
## ➤ 发展脉络



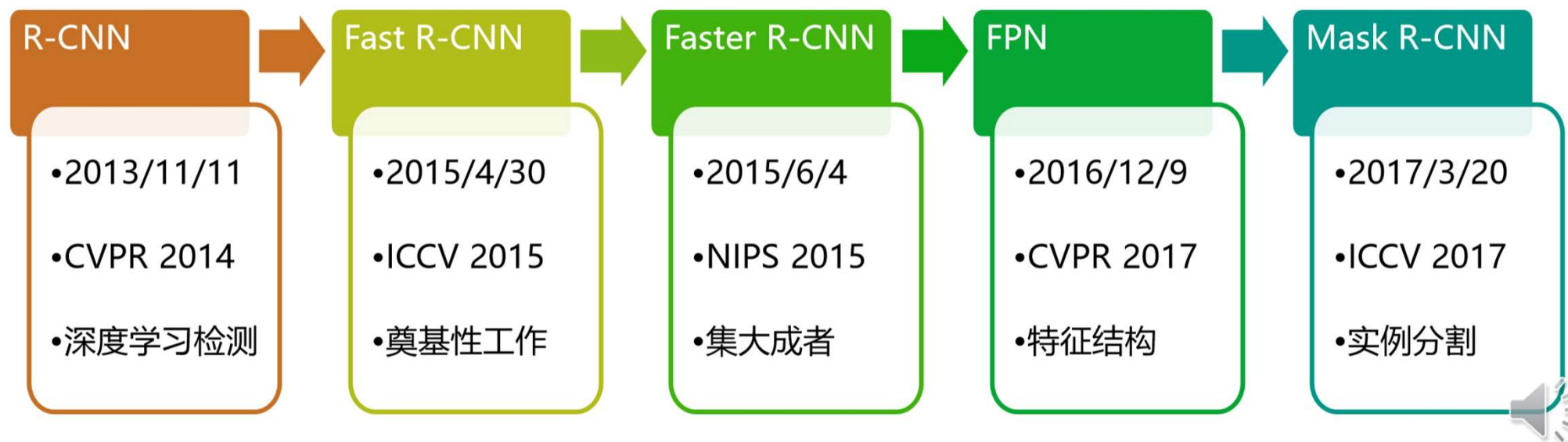
目标检测论文数量的年度变化趋势



## ➤ 发展脉络

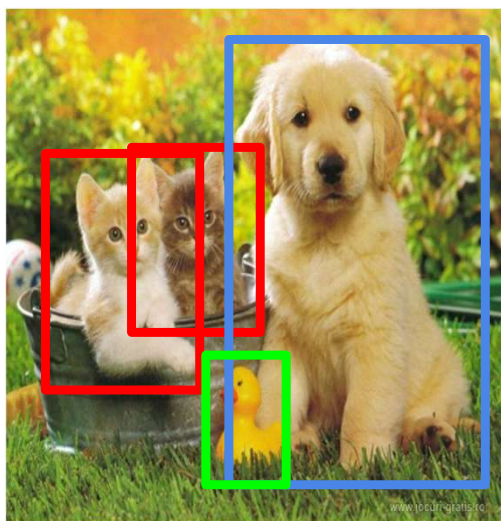


- R-CNN系列：从2013年的R-CNN出现，到2015年中旬的Faster R-CNN成熟，再到后来的开枝散叶



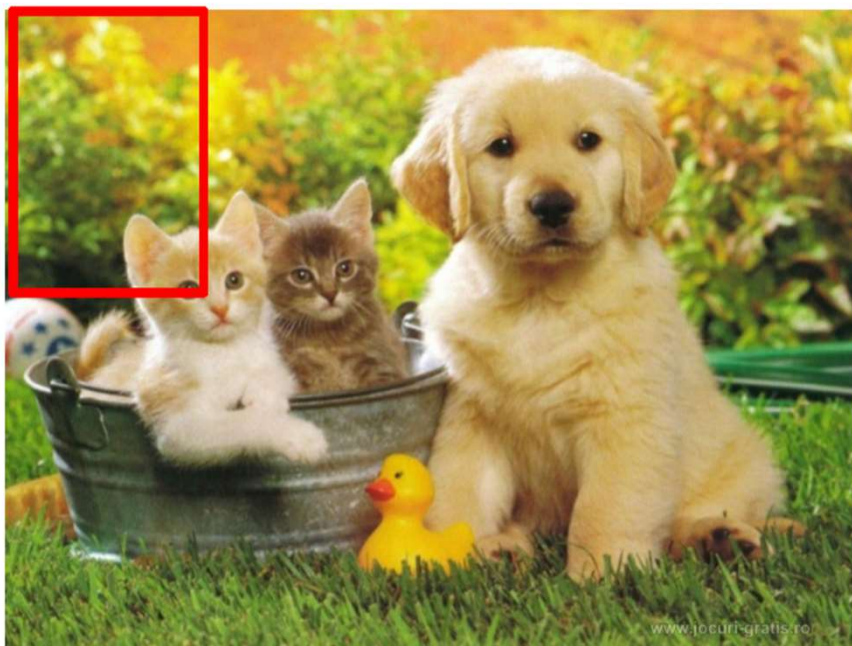


- 对图像中所有物体均预测其类别和目标框



CAT, DOG, DUCK

- 一种简单思路：将目标检测视为分类问题



Classes = [cat, dog, duck]

Cat ? NO

Dog ? NO

Duck? NO

- 一种简单思路：将目标检测视为分类问题



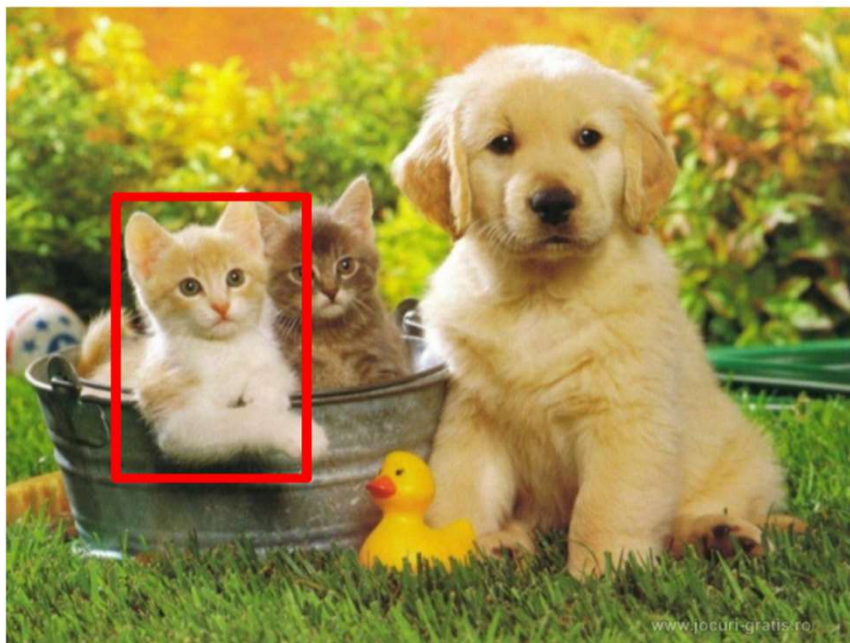
Classes = [cat, dog, duck]

Cat ? NO

Dog ? NO

Duck? NO

- 一种简单思路：将目标检测视为分类问题



Classes = [cat, dog, duck]

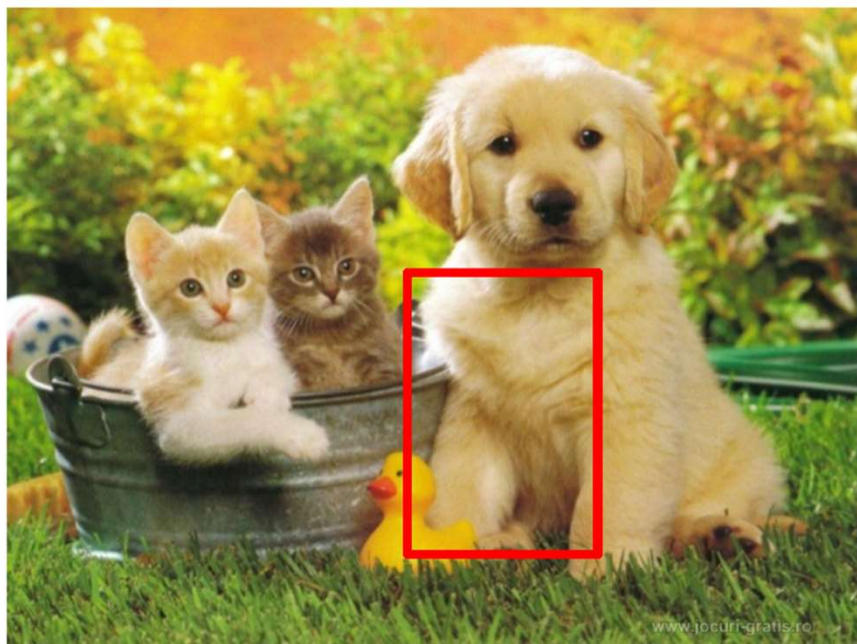
Cat ? YES

Dog ? NO

Duck? NO



- 一种简单思路：将目标检测视为分类问题



Classes = [cat, dog, duck]

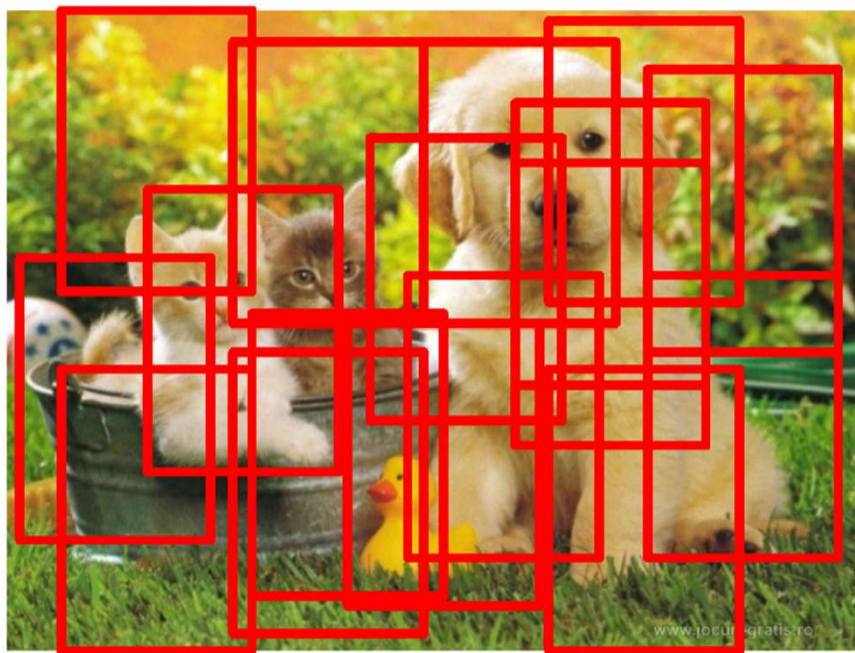
Cat ? NO

Dog ? NO

Duck? NO



➤ 一种简单思路：将目标检测视为分类问题

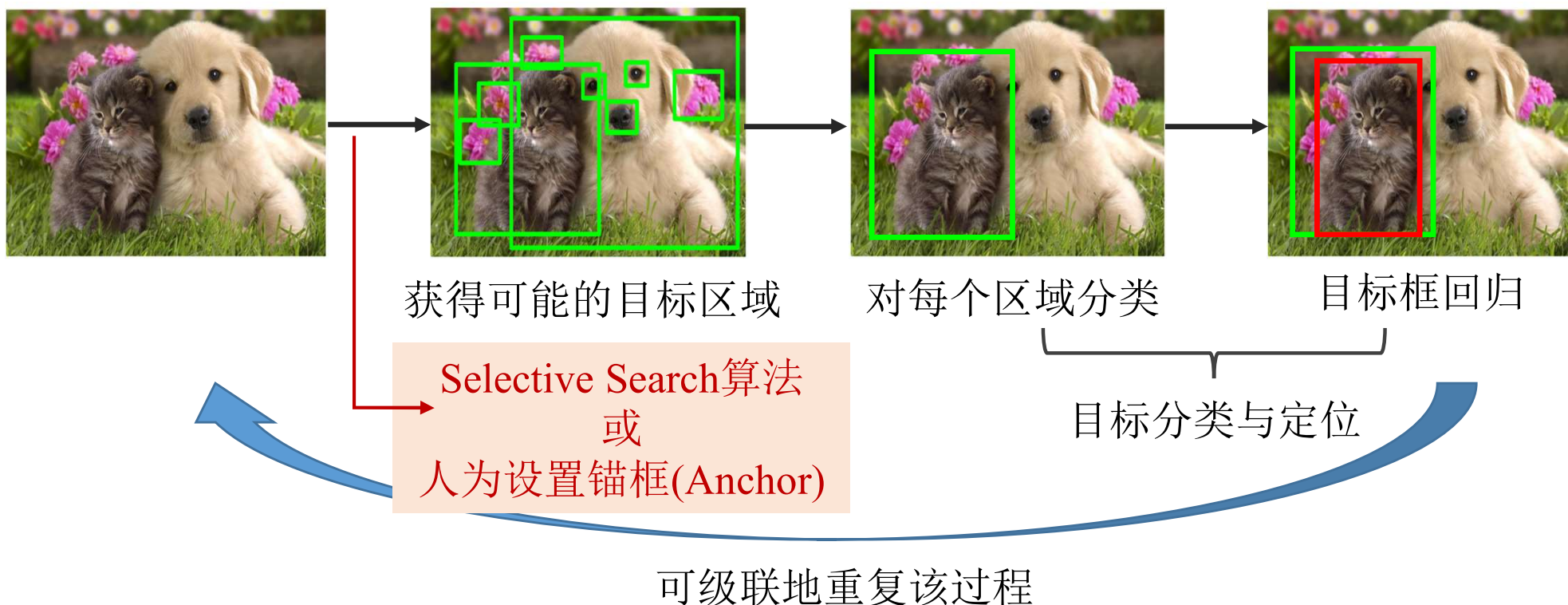


用不同尺度大小的框在所有位置进行遍历测试

计算量巨大！

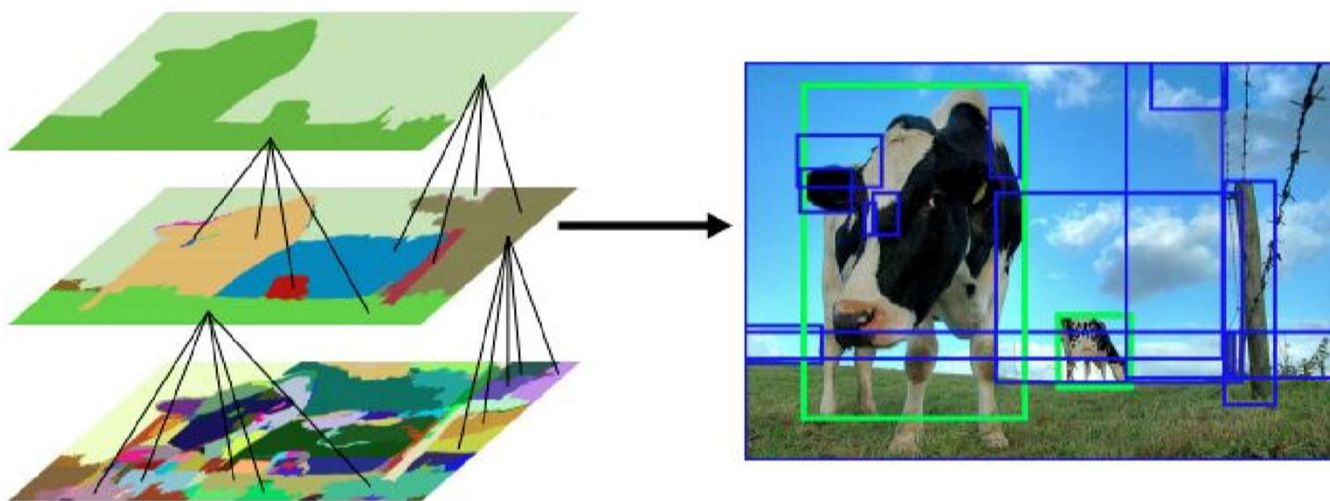
## ➤ Region Proposals

- 首先找到或设定图像中可能存在物体的所有区域
- 再对这些区域进行检测、分类



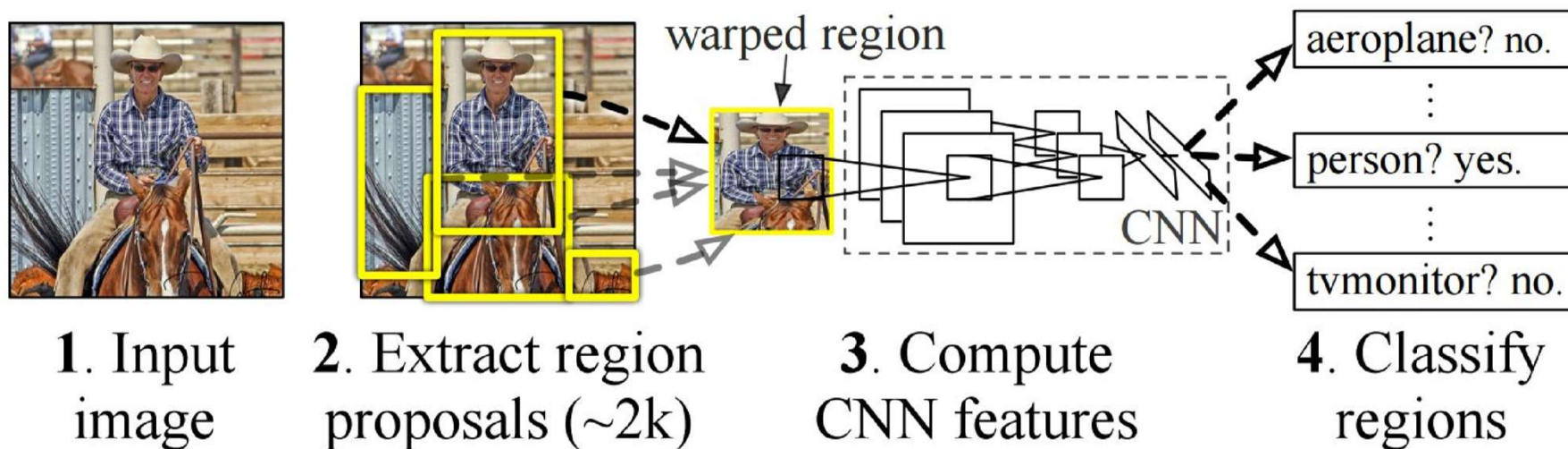
## ➤ Selective Search (SS)算法

- 利用图像分割产生初始分割区域 -> 利用相似度进行区域合
- 如何计算两个区域的相似度?
  - 计算颜色、纹理、大小和形状交叠的差异，用不同的权重相加



Uijlings et al. Selective search for object recognition. IJCV 2013

## ➤ 基于CNN的目标检测算法：R-CNN



使用SS算法，在输入图像上生成~2K个候选区域

每个候选区域输入CNN中提取特征

把特征送入每一类的SVM分类器，进行类别判断

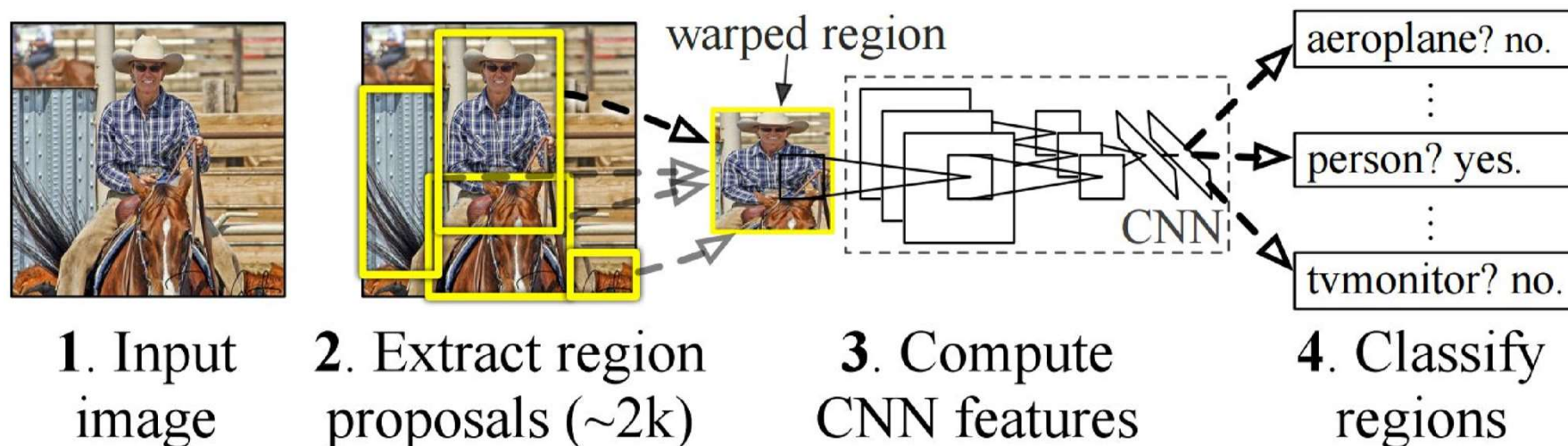
使用回归器精细修正候选框位置

Girshick et al. Rich feature hierarchies for accurate object detection and semantic segmentation. CVPR 2014





## ➤ 基于CNN的目标检测算法：R-CNN



### R-CNN的缺点:

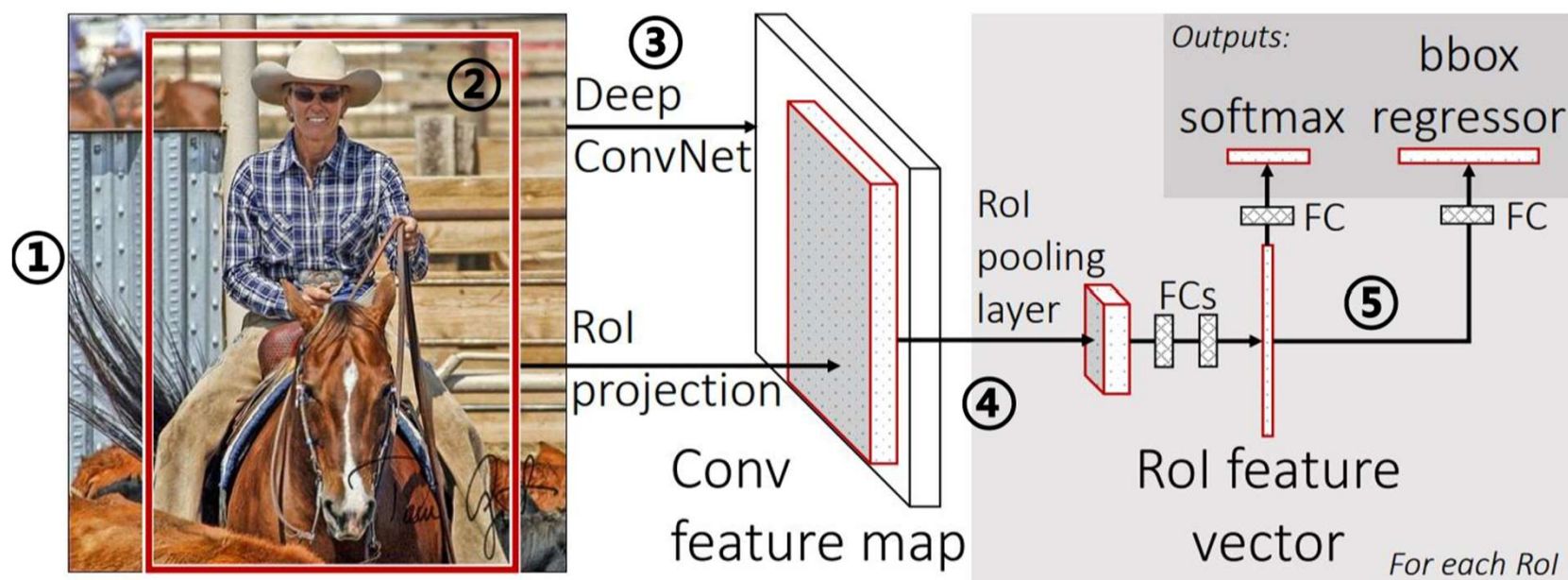
- 使用Selective Search生成候选区域非常**耗时**
- 一张图像~2K个候选区域，需~2K次CNN提取特征，**存在大量重复计算**
- 特征提取、图像分类、边框回归是三个独立步骤，需**分别训练**，**测试效率低**

**Fast R-CNN算法**



Girshick Fast R-CNN. ICCV 2015

## ➤ Fast R-CNN算法

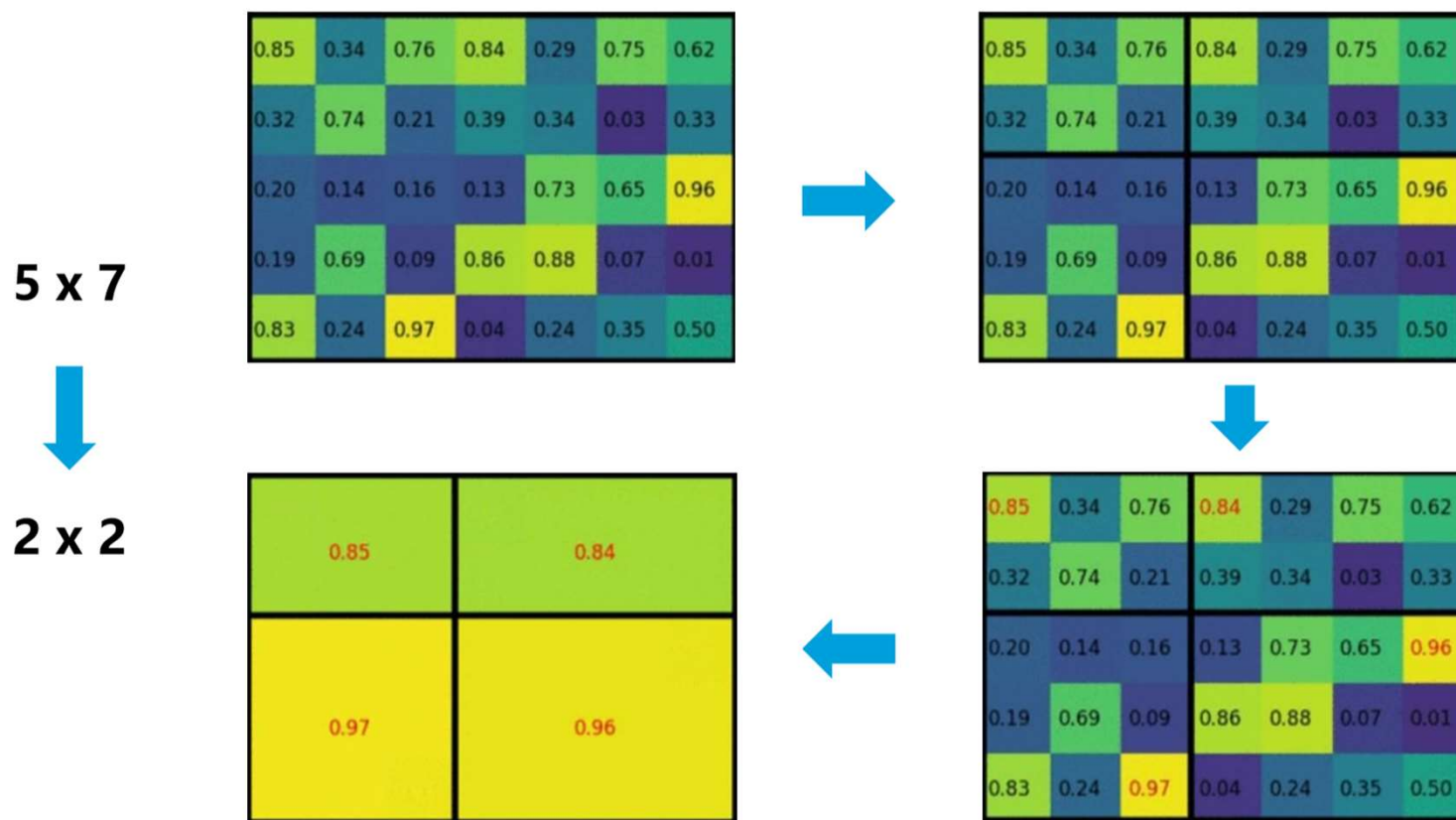


- ② 候选区域生成：使用Selective Search算法，在输入图像上生成~2K个候选区域
- ③ **特征提取**：将整张图像传入CNN提取特征
- ④ 候选区域特征：利用RoIPooling分别生成每个候选区域的特征
- ⑤ 候选区域分类和回归：利用扣取的特征，对每个候选区域进行分类和回归



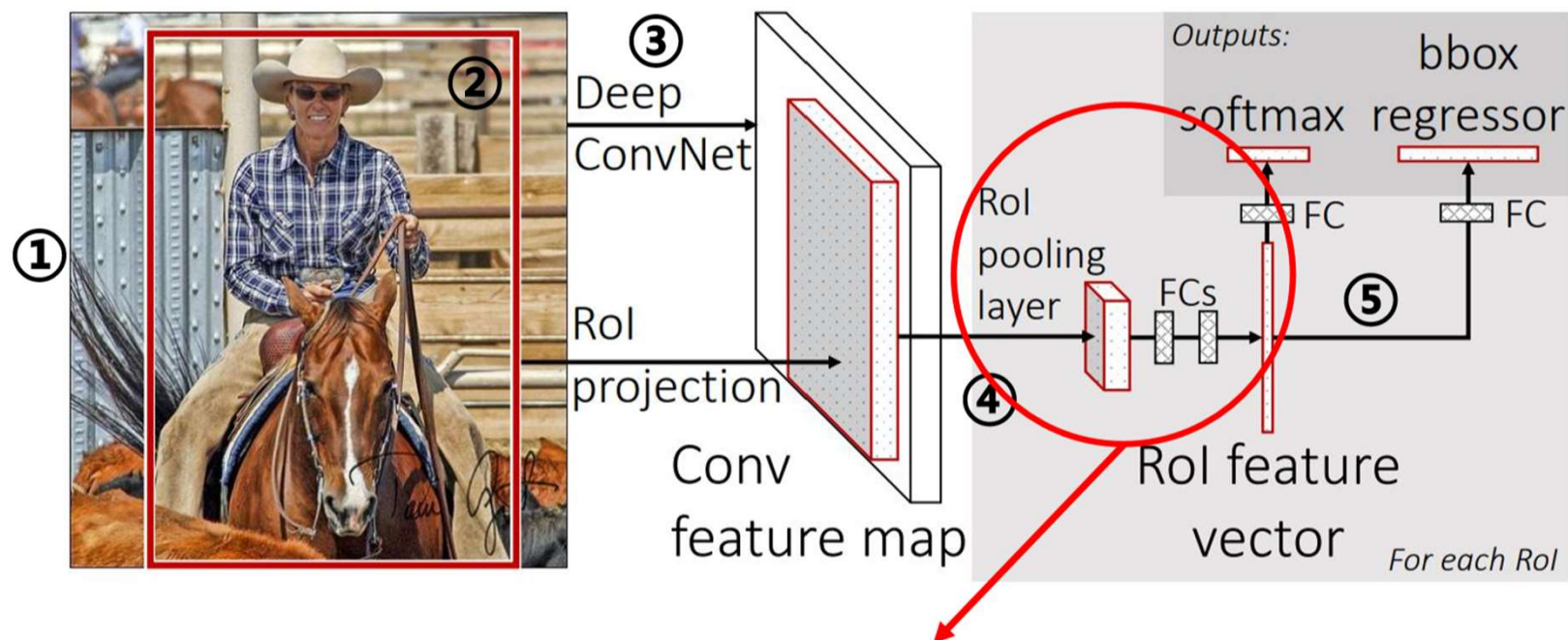
## ➤ Fast R-CNN算法

- RoIPooling: 利用特征采样, 把不同空间大小的特征, 变成空间大小一致的特征



Girshick Fast R-CNN. ICCV 2015

## ➤ Fast R-CNN算法



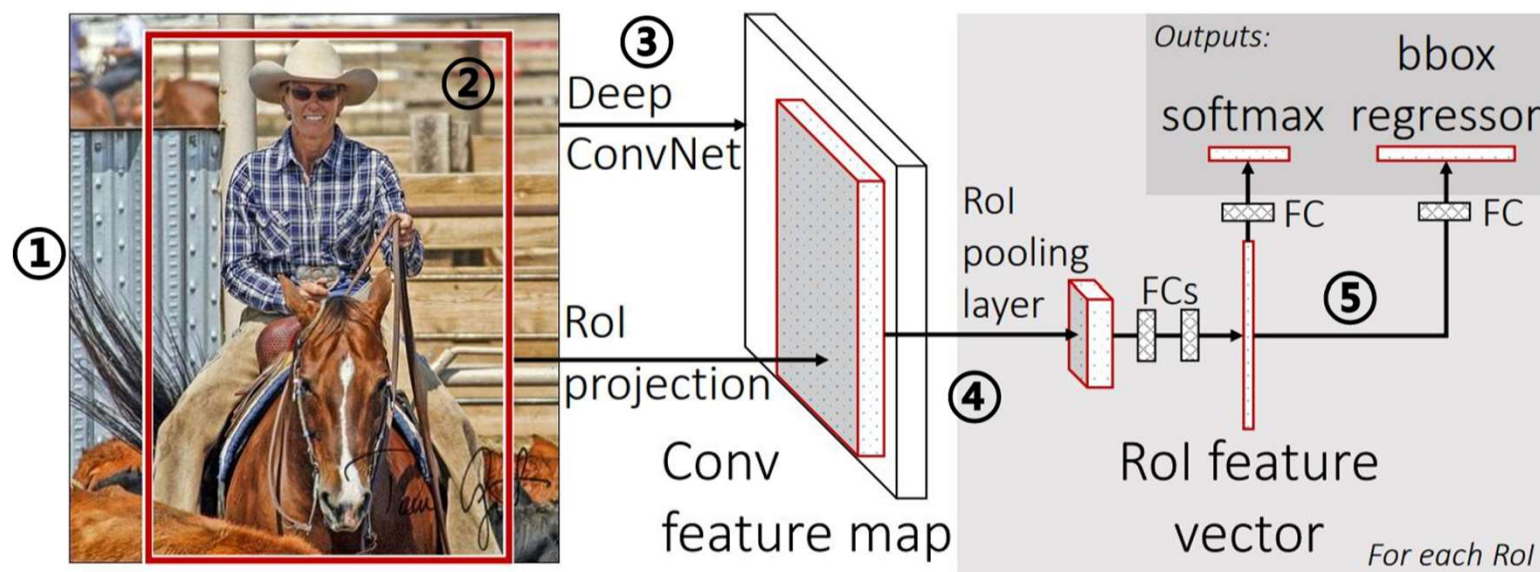
问：为什么要用RoIPooling把不同大小的特征变成固定大小？

- 网络后面是全连接层（FC层），要求输入有固定的维度
- 各个候选区域的特征大小一致，可以组成batch进行处理



Girshick Fast R-CNN. ICCV 2015

## ➤ Fast R-CNN算法



与R-CNN算法相比，Fast R-CNN的改进点：

- 可以端到端的多任务训练
- Fast R-CNN比R-CNN快了200多倍，并且精度更高

**Faster R-CNN算法**

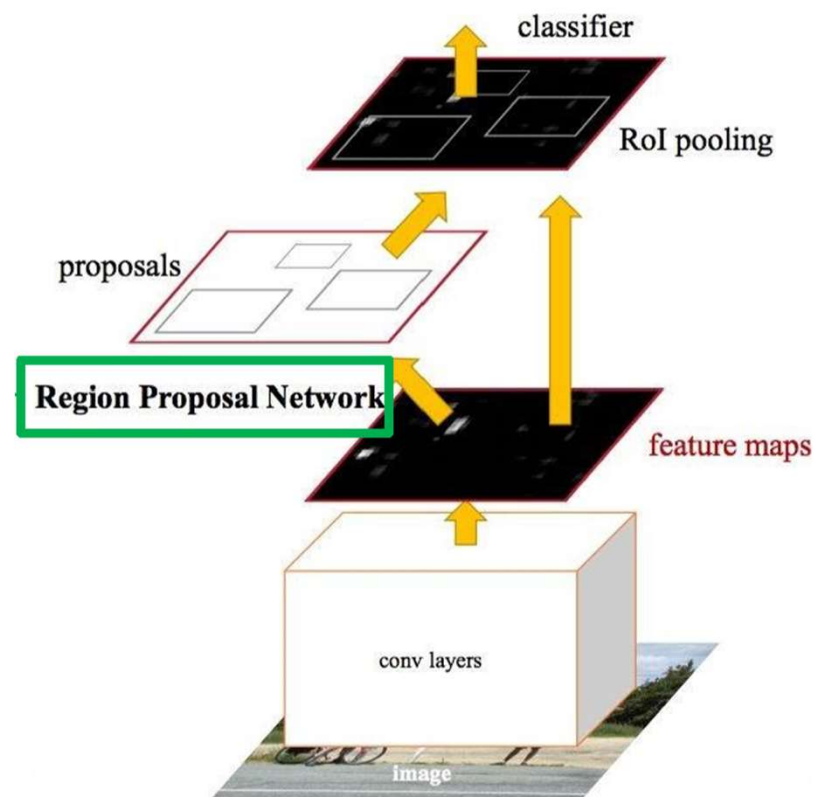
存留问题：生成候选区域算法（Selective Search）非常慢（耗时2s）



## ➤ Faster R-CNN算法

Fast R-CNN = **Selective Search** + Fast R-CNN

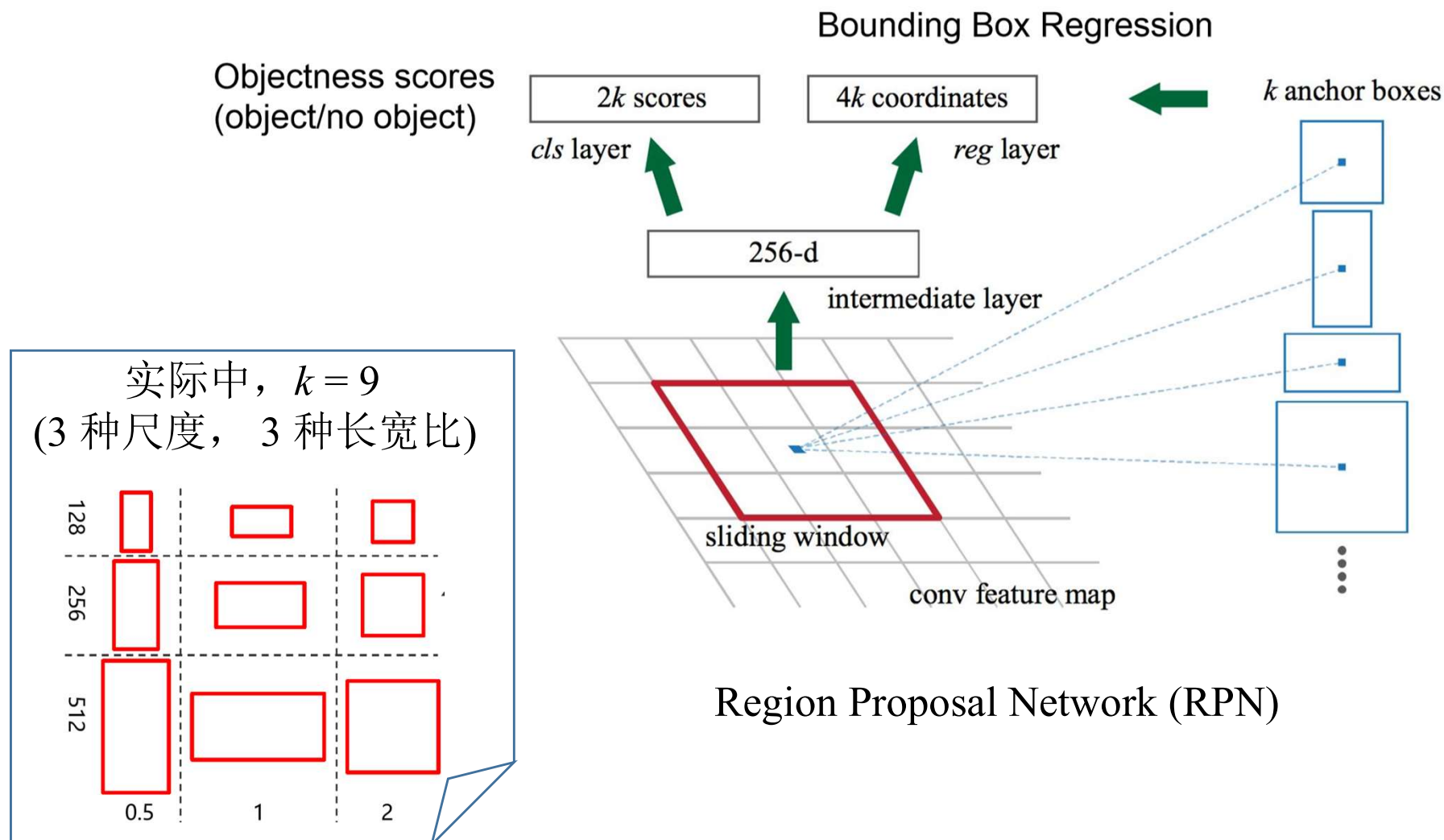
Faster R-CNN = **RPN** + Fast R-CNN



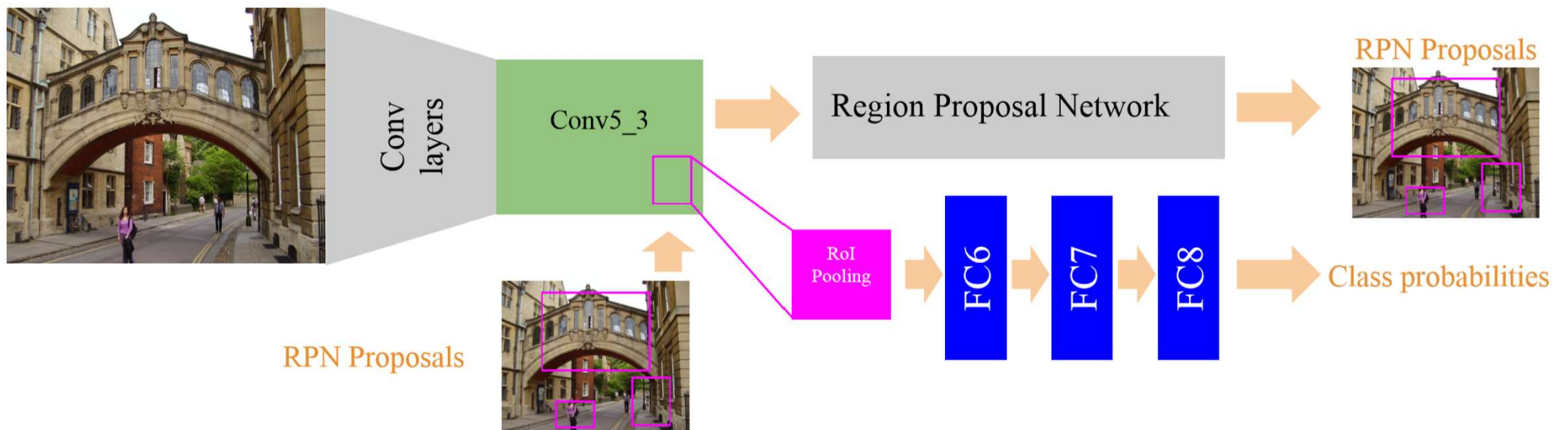
Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks, NIPS 2015



## ➤ Faster R-CNN算法



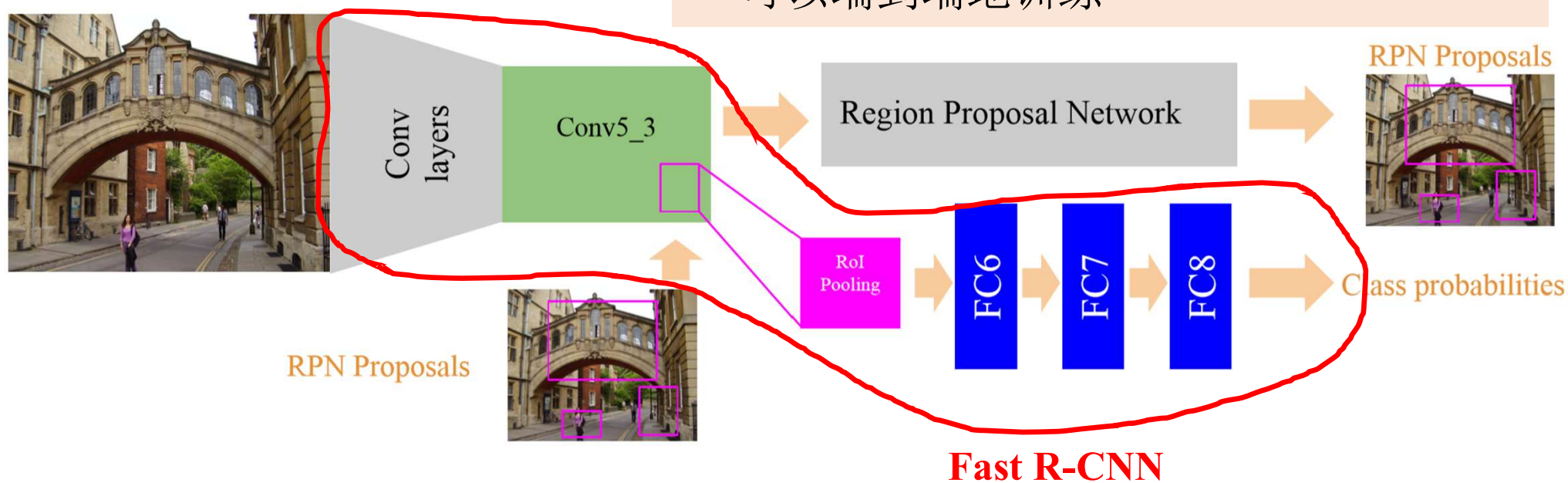
## ➤ Faster R-CNN算法



Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks, NIPS 2015

## ➤ Faster R-CNN算法

- RPN取代了耗时的Selective Search
- RPN与Fast RCNN共享卷积层
- 引入计算量小，耗时少
- 可以端到端地训练



Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks, NIPS 2015



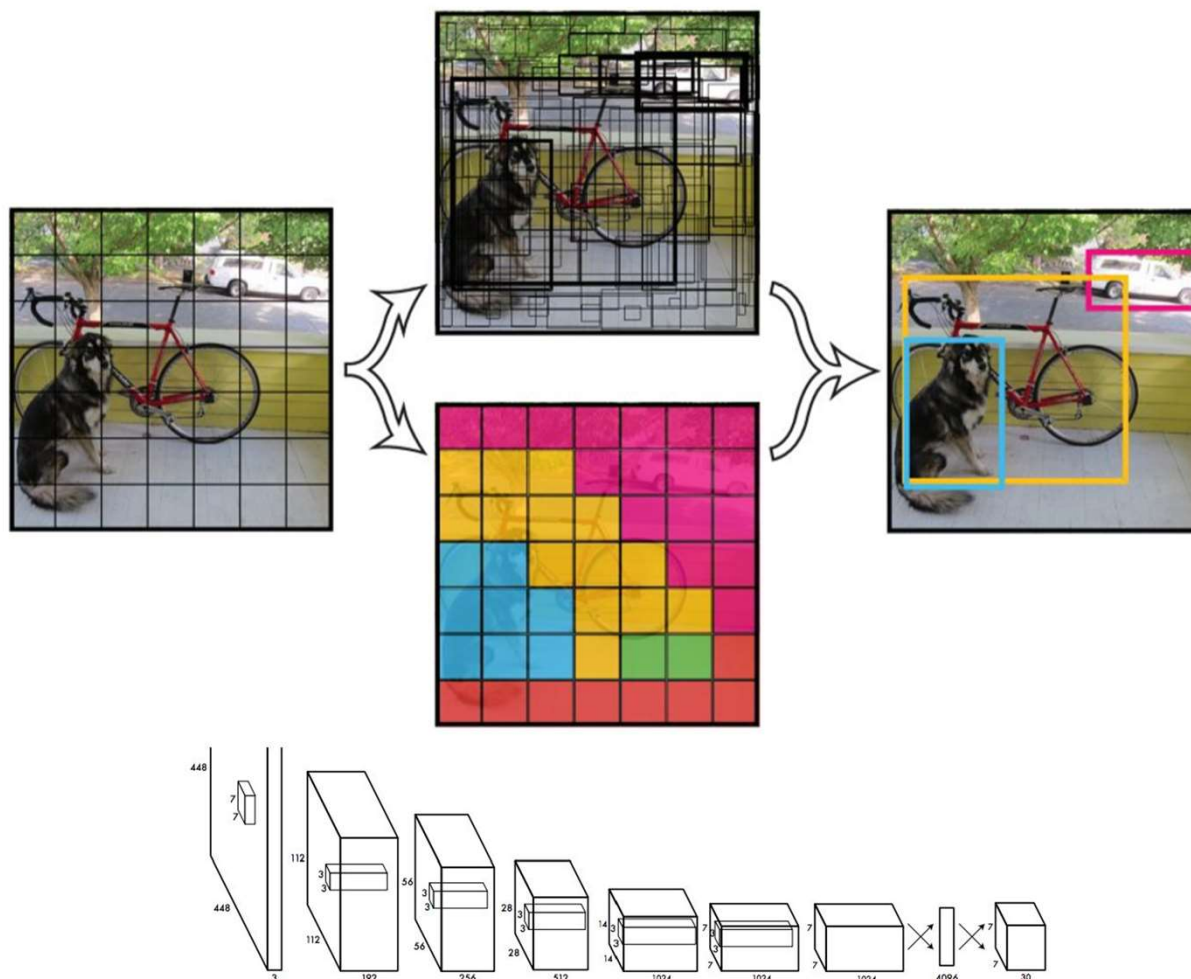


## ➤ 性能对比

|                                         | <b>R-CNN</b> | <b>Fast R-CNN</b> | <b>Faster R-CNN</b> |
|-----------------------------------------|--------------|-------------------|---------------------|
| Test time per image<br>(with proposals) | 50 seconds   | 2 seconds         | <b>0.2 seconds</b>  |
| (Speedup)                               | 1x           | 25x               | <b>250x</b>         |
| mAP (VOC 2007)                          | 66.0         | <b>66.9</b>       | <b>66.9</b>         |



## ➤ 单阶段目标检测算法-YOLO系列



Redmon et al. You Only Look Once: Unified, Real-Time Object Detection, CVPR 2016



## ➤ 常用数据库

### 通用物体检测数据集

- PASCAL VOC
- MS COCO
- OpenImages
- LVIS

### 人脸检测数据集

- AFW
- PASCAL FACE
- MAF
- MAFA
- Fddb
- WIDER FACE

### 行人检测数据集

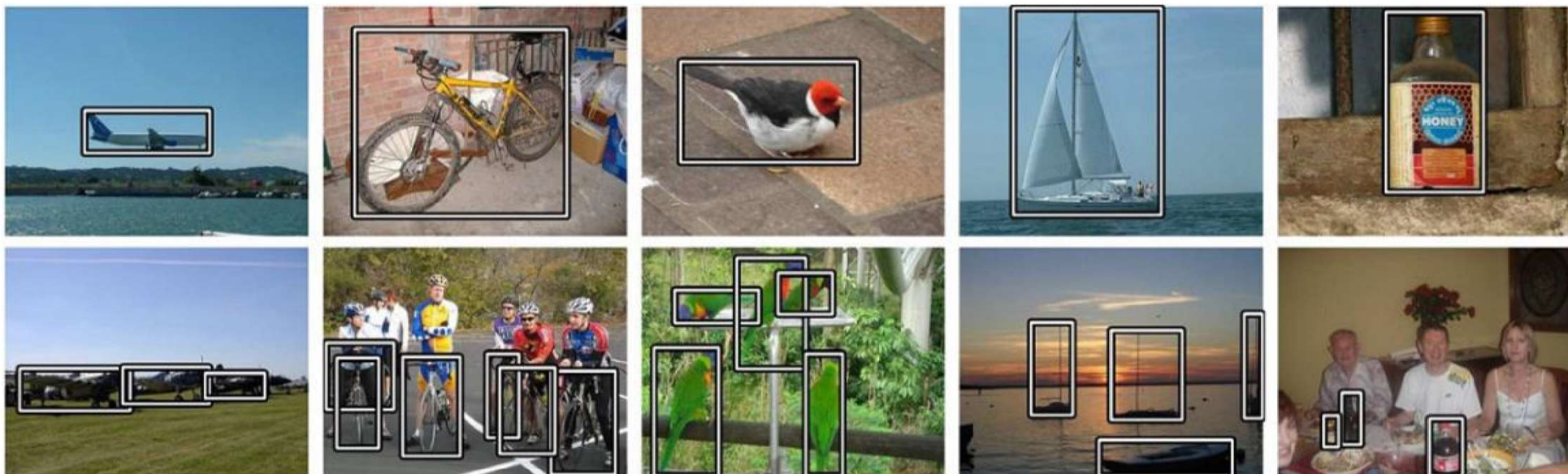
- Caltech-USA
- CityPersons
- CrowdHuman
- WiderPerson
- EuroCityPersons

人脸检测数据集: WIDER FACE



## ➤ 通用物体检测数据库- PASCAL VOC

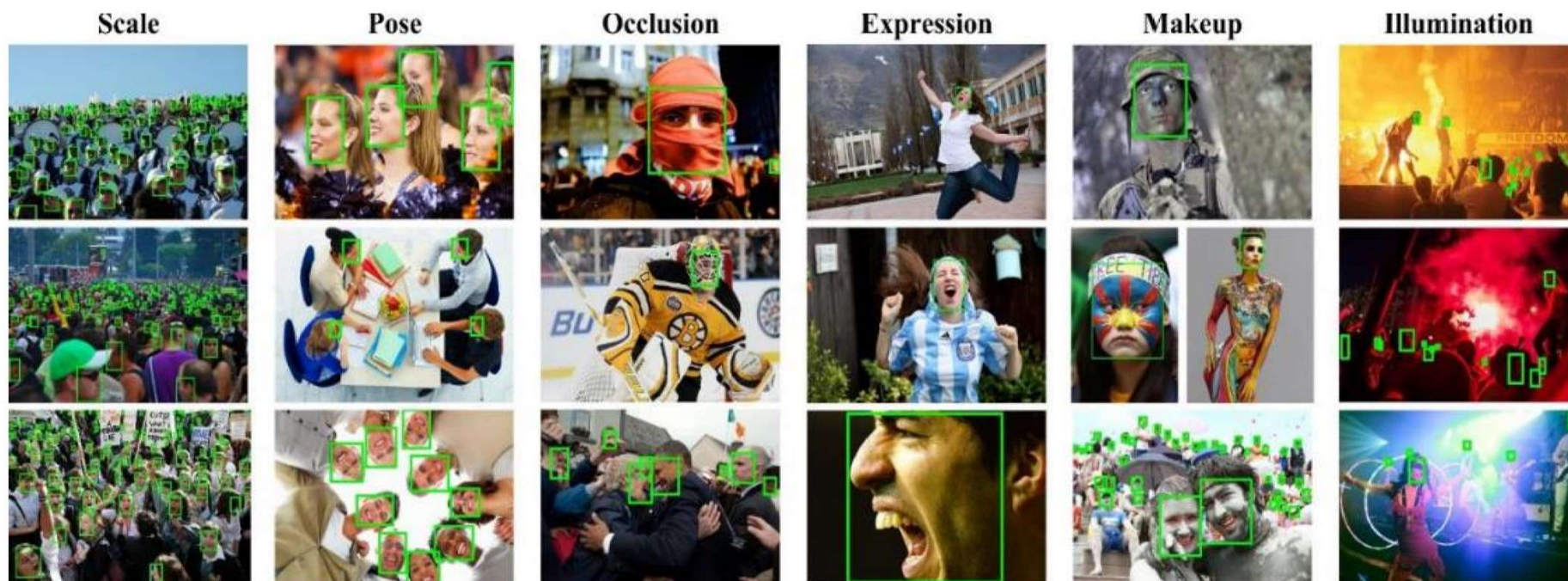
- 从2005 年开始举办的著名竞赛，目标检测为其中的一项比赛项目
- **PASCAL VOC 2007**: 9963张图像，24640 个标注
- **PASCAL VOC 2012**: 11530 张图像，27450个标注





## ➤ 人脸检测数据库-WIDER FACE

- 32203张图片、393703张人脸
- 训练集12880、验证集3226、测试集16097
- 人脸检测最大最权威的数据集，难度非常大



## ➤ 行人检测数据库-WiderPerson

- 13382张图，约40万个标注，丰富的场景
- 训练集8000 张、验证集1000 张、测试集4382 张





PART THREE

## 卷积神经网络实践

## CNN Practice





## ➤ 两种任务（任选其一）

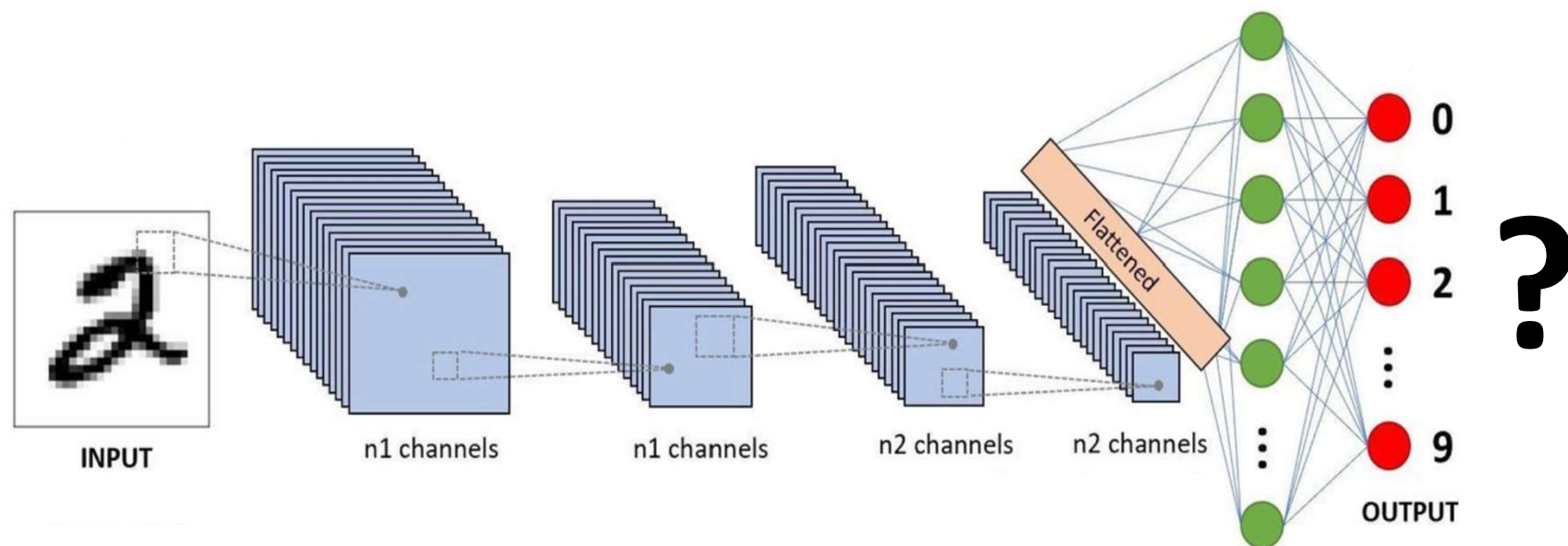
- 图像分类
- 目标检测





## ➤ 任务描述

- 训练一个手写字体的**CNN**分类模型，使其能够准确地预测输入图像的数字类别（**10**分类问题）



## ➤ 数据库

- 采用MNIST数据库



## ➤ 评估准则

- 分类准确率 (Accuracy)

以3类为例的混淆矩阵

|              |         | Prediction |         |         |
|--------------|---------|------------|---------|---------|
|              |         | Class 1    | Class 2 | Class 3 |
| Ground Truth | Class 1 | x(1,1)     | x(1,2)  | x(1,3)  |
|              | Class 2 | x(2,1)     | x(2,2)  | x(2,3)  |
|              | Class 3 | x(3,1)     | x(3,2)  | x(3,3)  |

$$Accuracy = \frac{\sum_{i=1}^3 x(i, i)}{\sum_{i=1}^3 \sum_{j=1}^3 x(i, j)}$$

对角线为正确分类，其余位置为错分类

## ➤ 任务描述

- 针对现有公开的**Faster R-CNN**代码，进行相关环境与调试
- 利用训练好的公开模型，完成对如下**5**张图像的测试。图像链接：

<https://github.com/rbgirshick/py-faster-rcnn/tree/master/data/demo>





## ➤ 任务描述

- 针对现有公开的**Faster R-CNN**代码，进行相关环境环境与调试
- 利用训练好的公开模型，完成对如下**5**张图像的测试。图像链接：  
<https://github.com/rbgirshick/py-faster-rcnn/tree/master/data/demo>
- 利用训练好的公开模型，对**MS COCO**验证集上进行评测，得到官方给出的**平均精度均值 (mAP)**

论文链接：

<https://arxiv.org/pdf/1506.01497.pdf>

推荐代码链接：

<https://github.com/facebookresearch/maskrcnn-benchmark>

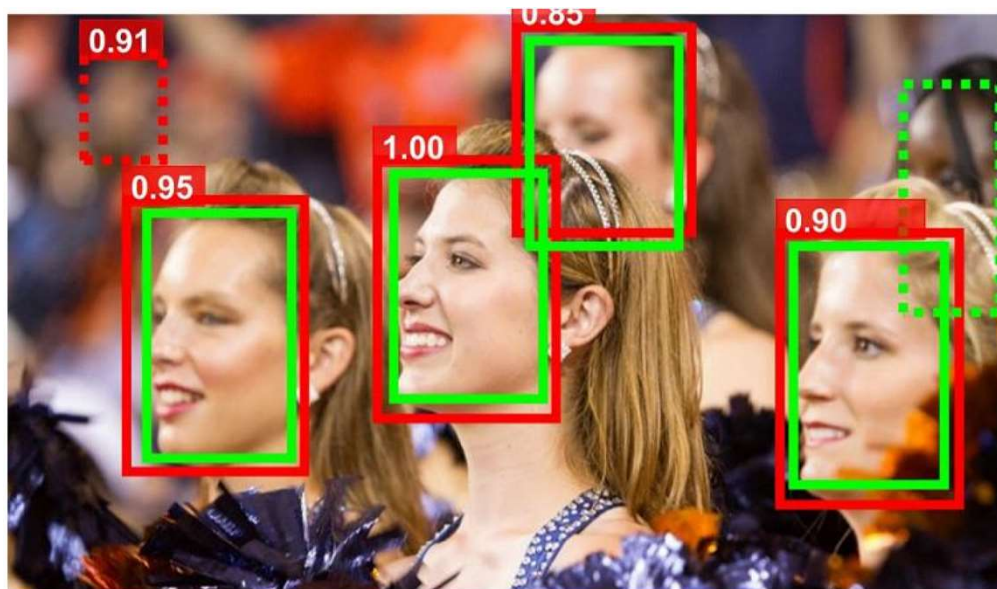
<https://github.com/rbgirshick/py-faster-rcnn/tree/master/>

<https://github.com/jwyang/faster-rcnn.pytorch>



## ➤ 评估指标

- 通用物体目标检测 (20类、80类、1000类)
- 检测精度评价指标：每类先单独计算评价指标，然后多类之间求平均
- 以单类物体检测为例来说明

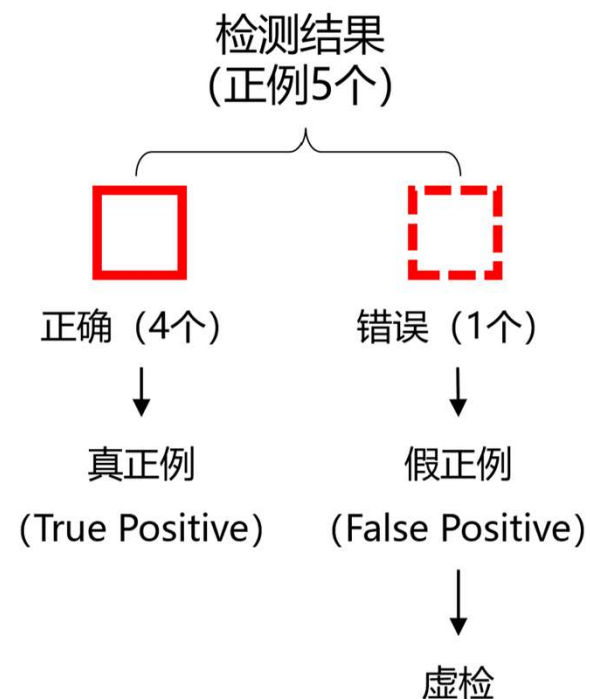
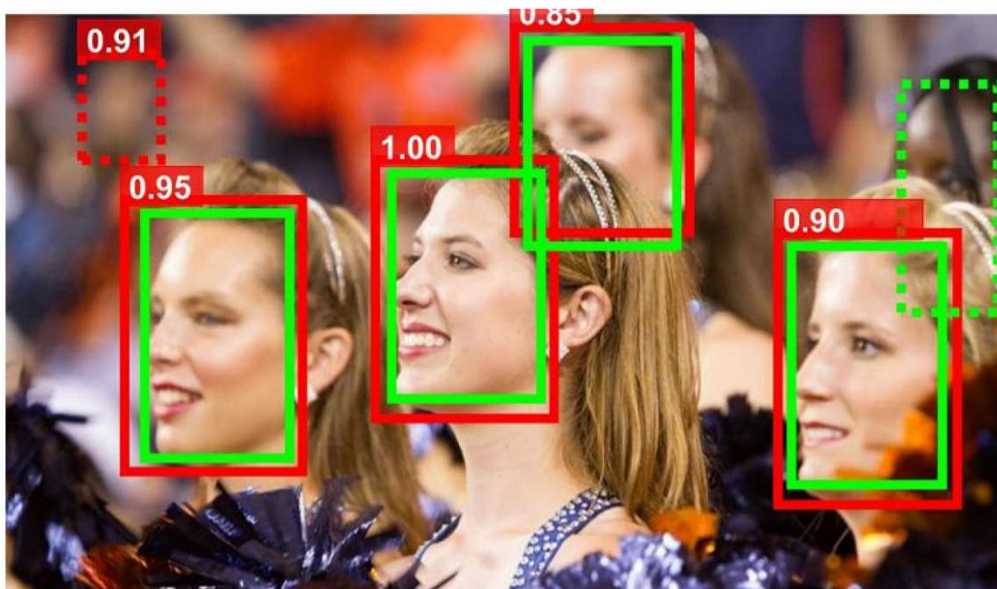


- 绿色实线或虚线框：真实标注
- 红色实线或虚线框：预测检测结果
- 红色数字：检测结果为人脸的概率


$$\text{交并重叠比 (IoU)} = \frac{\text{交集的面积}}{\text{并集的面积}} \geq 0.5$$

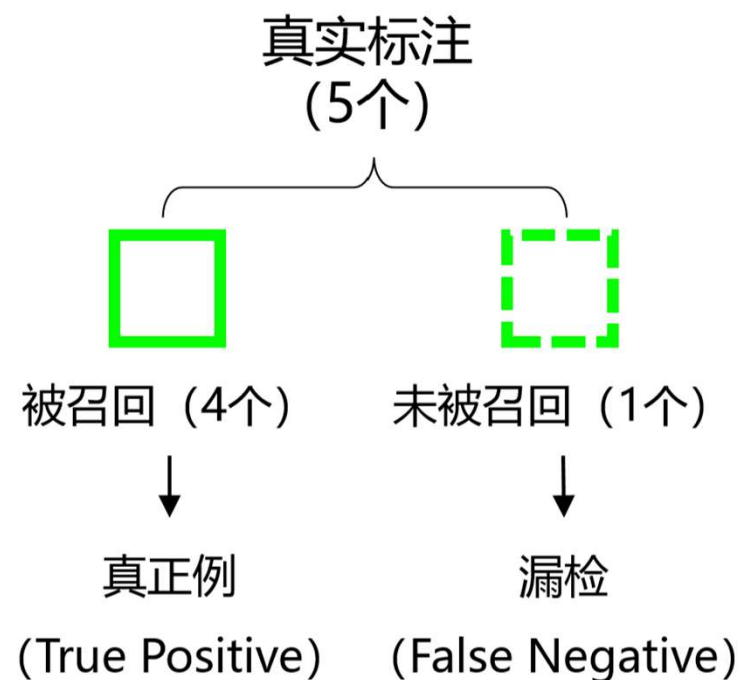
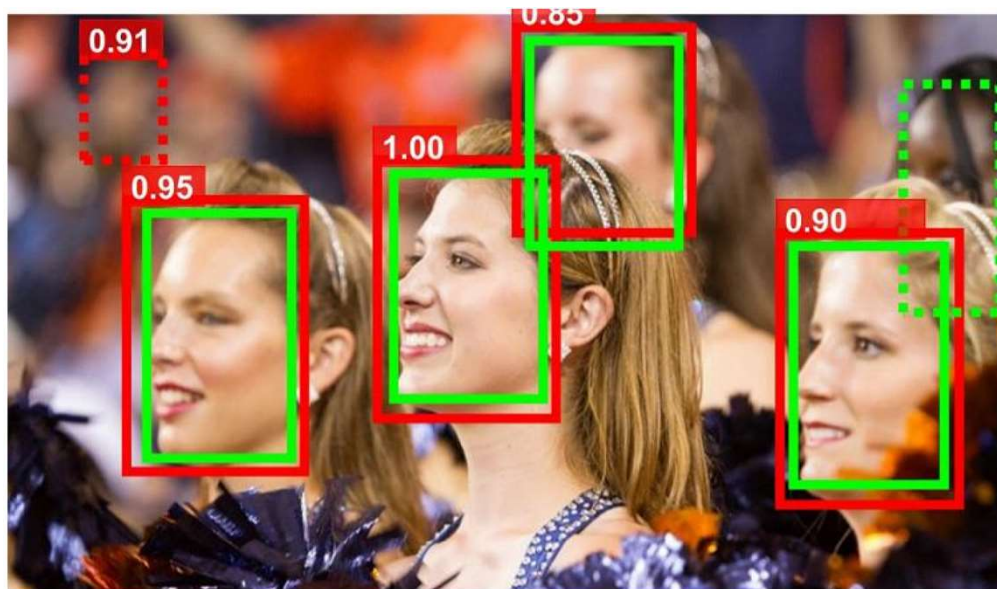
## ➤ 评估指标

- 通用物体目标检测 (20类、80类、1000类)
- 检测精度评价指标: 每类先单独计算评价指标, 然后多类之间求平均
- 以单类物体检测为例来说明



## ➤ 评估指标

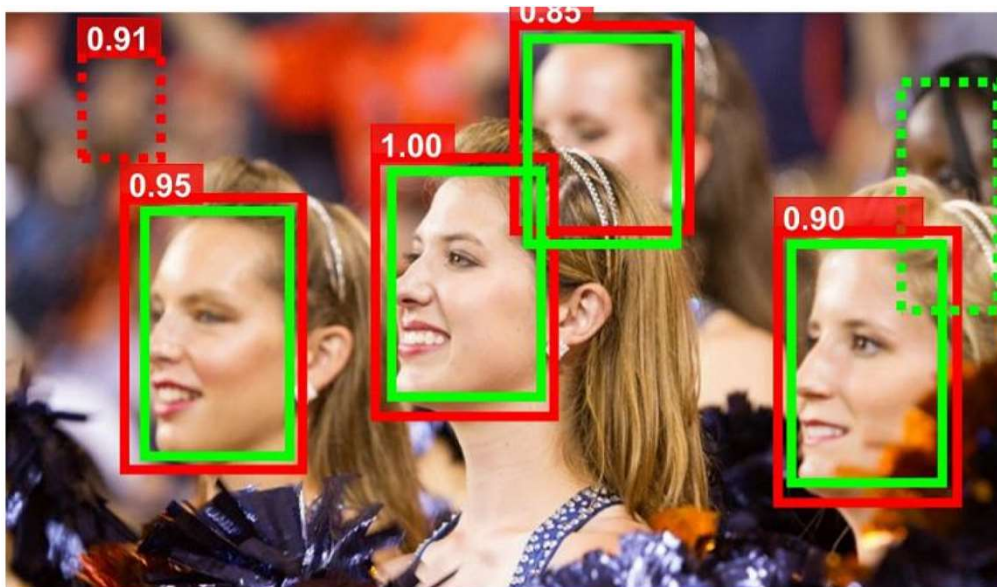
- 通用物体目标检测 (20类、80类、1000类)
- 检测精度评价指标: 每类先单独计算评价指标, 然后多类之间求平均
- 以单类物体检测为例来说明





## ➤ 召回率 (Recall Rate)

- 通用物体目标检测 (20类、80类、1000类)
- 检测精度评价指标: 每类先单独计算评价指标, 然后多类之间求平均
- 以单类物体检测为例来说明



检测结果(□□)= 5个    真实标注 (□□) = 5个

被召回的真实标注(□) = 真正例 (□) = 4个

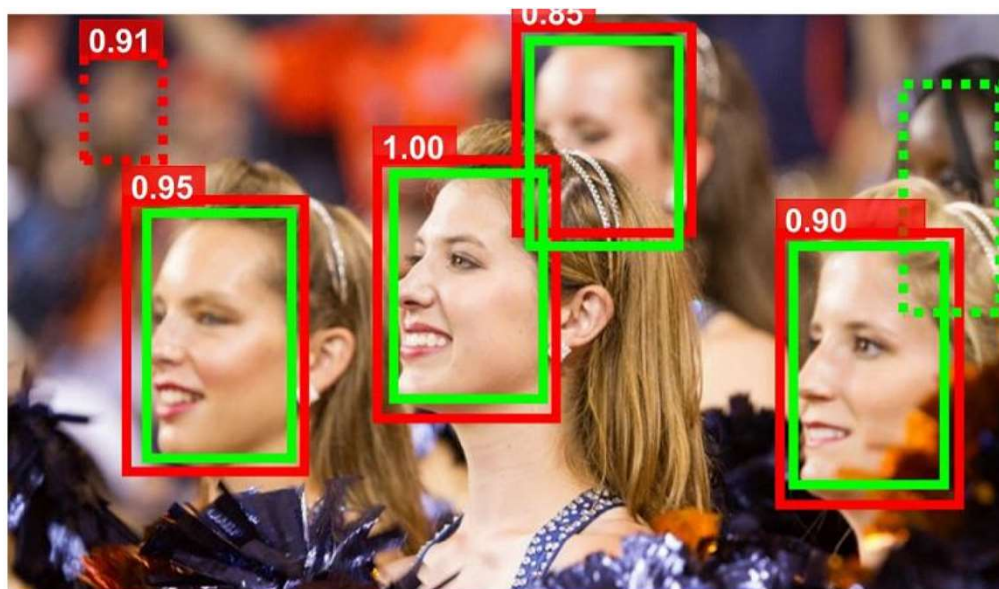
虚检(□□)= 1个

漏检 (□□) = 1个

$$\begin{aligned}\text{召回率} &= \frac{\text{被召回的真实标注的数量}}{\text{真实标注的数量}} \\ &= \frac{4}{5} = 80\%\end{aligned}$$

## ➤ 精度 (Precision)

- 通用物体目标检测 (20类、80类、1000类)
- 检测精度评价指标: 每类先单独计算评价指标, 然后多类之间求平均
- 以单类物体检测为例来说明



检测结果(□) = 5个      真实标注 (□) = 5个

被召回的真实标注(□) = 真正例 (□) = 4个

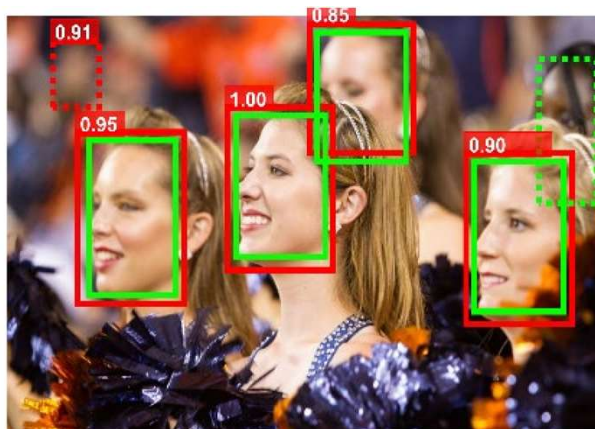
虚检(□) = 1个

漏检 (□) = 1个

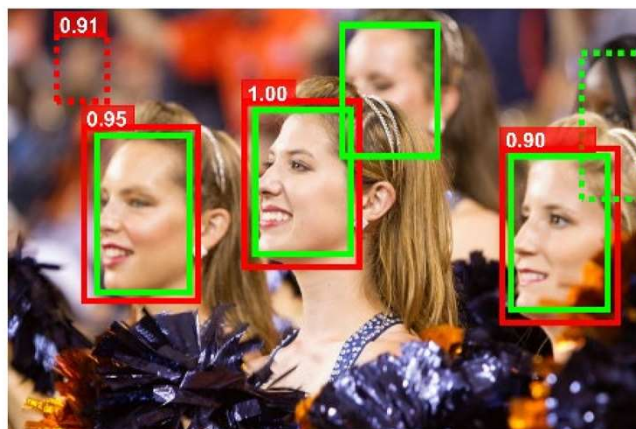
$$\text{精度} = \frac{\text{真正例的数量}}{\text{检测结果的数量}} = \frac{4}{5} = 80\%$$



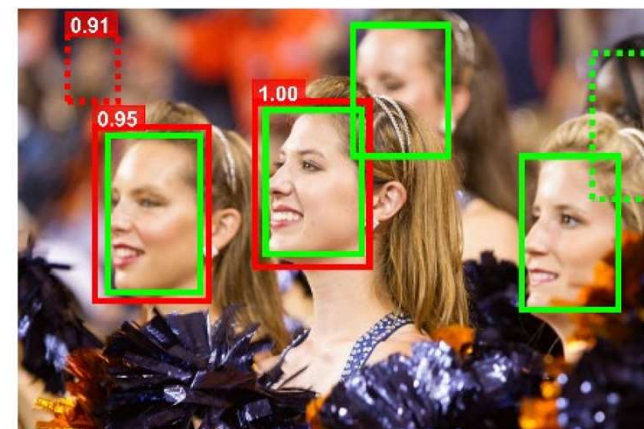
## ➤ 召回率与精度



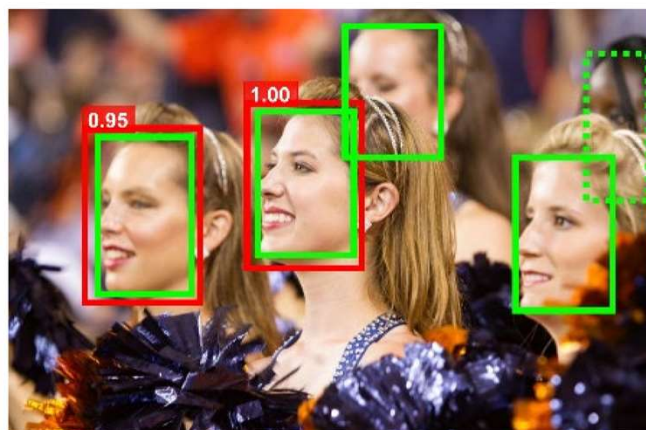
召回率(80%), 精度(80%)



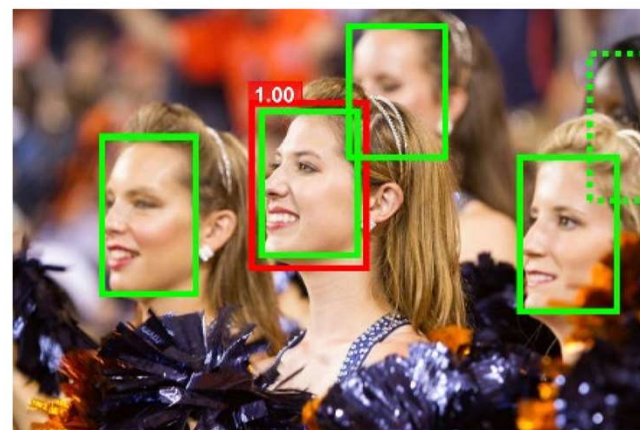
召回率(60%), 精度(75%)



召回率(40%), 精度(67%)



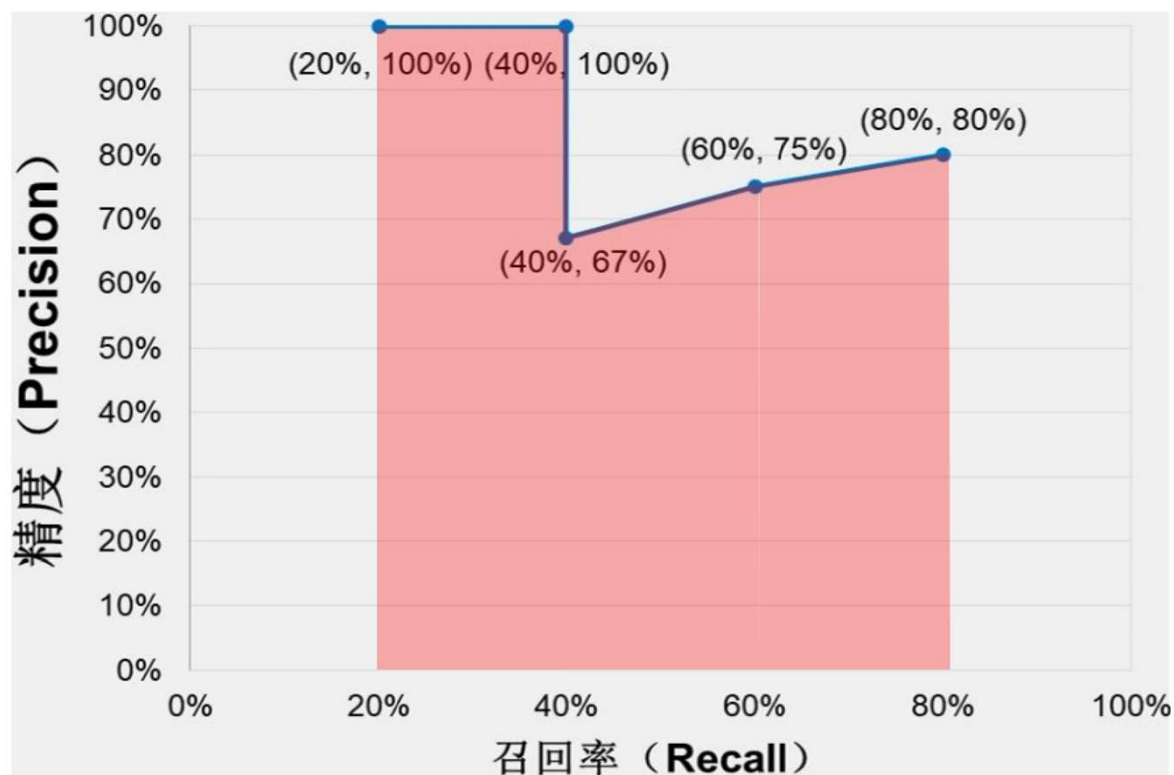
召回率(40%), 精度(100%)



召回率(20%), 精度(100%)



## ➤ 平均精度均值 (mAP)



N个不同 $IoU$ 阈值下的召回率-精度曲线

$IoU \geq 0.5$ , 常用评价指标

### 平均精度 (AP)

精度-召回率曲线下的面积

### 平均精度均值 (mAP)

不同类别的平均精度的均值

MS COCO数据集用的评价指标

- $IoU \geq [0.5, 0.55, 0.6, 0.65, 0.7, 0.75, 0.8, 0.85, 0.9, 0.95]$